

Smart Campus Parking and Traffic Management System

CSA09 – Programming in Java
AWT/Swing + JDBC (MySQL)

Department of Computer Science and Engineering

Team Members

Name	Register Number	Role
KANIGASRI M	192565010	Database Design, JDBC & DAO Layer
CHAITHIKA L S	192311092	GUI Design, MainFrame & Entry/Exit
BABY SHALINI	192521268	Reservations, Passes, Violations, Reports & Documentation

GitHub Repository Links

Member	GitHub Repository
KANIGASRI M	https://github.com/KANIGASRI31/CSA0925-PROGRAMMING-IN-JAVA/tree/main/ASSIGNMENT
CHAITHIKA L S	https://github.com/chaithika09/Java-Assignment-chaithika
BABY SHALINI	https://github.com/192521268simats-cell/java-

PART 1 – PSEUDOCODE

1. Database Connection Module

```
BEGIN DBConnection
  CONSTANT URL <- "jdbc:mysql://localhost:3306/campus_parking"
  CONSTANT USER <- "root"
  VARIABLE conn <- NULL

  FUNCTION getConnection()
    IF conn is NULL OR conn is closed THEN
      LOAD com.mysql.cj.jdbc.Driver
      conn <- DriverManager.getConnection(URL, USER, PASSWORD)
    END IF
    RETURN conn
  END FUNCTION

  FUNCTION testConnection()
    TRY
      CALL getConnection()
      IF conn is valid THEN RETURN null
    CATCH SQLException -> RETURN error message
    END TRY
  END FUNCTION

  FUNCTION close()
    IF conn is not NULL THEN conn.close(); conn <- NULL
  END IF
  END FUNCTION
END DBConnection
```

2. User Management Module

```
BEGIN UserDao
  FUNCTION addUser(user)
    sql <- "INSERT INTO users(full_name,email,phone,user_type,id_number,address)
VALUES(?,?,?,?,?,?)"
    ps <- PreparedStatement(sql)
    SET parameters from user object
    EXECUTE ps -> RETURN rowsAffected > 0
  END FUNCTION

  FUNCTION getAllUsers()
    sql <- "SELECT * FROM users ORDER BY full_name"
    stmt <- Statement() // plain Statement for simple SELECT
    RETURN mapResultSetToList(stmt.executeQuery(sql))
  END FUNCTION

  FUNCTION searchUsers(keyword)
    sql <- "SELECT * FROM users WHERE full_name LIKE ? OR email LIKE ? OR
id_number LIKE ?"
    ps <- PreparedStatement(sql)
    SET all params <- "%" + keyword + "%"
    RETURN executeAndMap(ps)
  END FUNCTION

  FUNCTION updateUser(user)
    sql <- "UPDATE users SET
full_name=?,email=?,phone=?,user_type=?,id_number=?,address=? WHERE user_id=?"
    EXECUTE PreparedStatement -> RETURN rowsAffected > 0
  END FUNCTION

  FUNCTION deleteUser(userId)
    sql <- "DELETE FROM users WHERE user_id=?"
```

```

        EXECUTE PreparedStatement -> RETURN rowsAffected > 0
    END FUNCTION
END UserDao

```

3. Vehicle Management Module

```

BEGIN VehicleDAO
    FUNCTION addVehicle(vehicle)
        sql <- "INSERT INTO
vehicles(user_id,license_plate,vehicle_type,make,model,color) VALUES(?,?,?,?,?,?)"
        EXECUTE PreparedStatement -> RETURN success

    FUNCTION getVehicleByPlate(plate)
        sql <- "SELECT v.*, u.full_name AS owner_name FROM vehicles v
JOIN users u ON v.user_id=u.user_id WHERE v.license_plate=?"
        EXECUTE PreparedStatement -> RETURN Vehicle or null

    FUNCTION getVehiclesByUser(userId)
        sql <- "SELECT * FROM vehicles WHERE user_id=?"
        EXECUTE PreparedStatement -> RETURN list
END VehicleDAO

```

4. Parking Zone and Slot Module

```

BEGIN ParkingZoneDAO
    FUNCTION getAllZones()
        sql <- "SELECT z.*, (SELECT COUNT(*) FROM parking_slots s
WHERE s.zone_id=z.zone_id AND s.status='Available') AS available_slots
FROM parking_zones z ORDER BY z.zone_name"
        stmt <- Statement() // plain Statement
        RETURN executeAndMapZones(stmt, sql)
    END FUNCTION
END ParkingZoneDAO

BEGIN ParkingSlotDAO
    FUNCTION getAvailableSlots(zoneId)
        sql <- "SELECT * FROM parking_slots WHERE zone_id=? AND status='Available'"
        EXECUTE PreparedStatement -> RETURN available slot list

    FUNCTION updateSlotStatus(slotId, status)
        sql <- "UPDATE parking_slots SET status=? WHERE slot_id=?"
        EXECUTE PreparedStatement -> RETURN success
END ParkingSlotDAO

```

5. Reservation Module

```

BEGIN ReservationDAO
    FUNCTION addReservation(reservation)
        // Step 1: Conflict check using PreparedStatement
        checkSql <- "SELECT COUNT(*) FROM reservations WHERE slot_id=? AND
status='Active'
AND NOT (reserved_until<=? OR reserved_from>=?)"
        IF count > 0 THEN THROW "Conflicting reservation exists"

        // Step 2: Insert reservation
        sql <- "INSERT INTO
reservations(user_id,vehicle_id,slot_id,reserved_from,reserved_until,status)
VALUES(?,?,?,?,?, 'Active')"
        EXECUTE PreparedStatement with RETURN_GENERATED_KEYS

        // Step 3: Update slot status
        CALL slotDAO.updateSlotStatus(slotId, 'Reserved')
        RETURN generatedKey
    END FUNCTION

```

```

        FUNCTION cancelReservation(reservationId)
            UPDATE status -> 'Cancelled'
            UPDATE slot status -> 'Available'
        END FUNCTION
    END ReservationDAO

```

6. Vehicle Entry / Exit – Stored Procedures & CallableStatement

```

STORED PROCEDURE sp_vehicle_entry(vehicleId, slotId, userId) -> sessionId
    GET slotStatus FROM parking_slots WHERE slot_id = slotId
    IF slotStatus != 'Available' THEN RAISE ERROR "Slot not available"
    ELSE
        INSERT INTO parking_sessions VALUES(vehicleId, slotId, userId, NOW(),
'Active')
        UPDATE parking_slots SET status='Occupied'
        RETURN LAST_INSERT_ID()
    END IF
END PROCEDURE

STORED PROCEDURE sp_vehicle_exit(sessionId) -> fee
    GET entryTime, hourlyRate FROM sessions JOIN slots JOIN zones
    hours <- MAX(CEIL(minutes_diff / 60), 1)
    fee <- hours x hourlyRate
    UPDATE parking_sessions SET exit_time=NOW(), fee=fee, status='Completed'
    UPDATE parking_slots SET status='Available'
    RETURN fee
END PROCEDURE

BEGIN SessionDAO
    FUNCTION recordEntry(vehicleId, slotId, userId)
        cs <- CallableStatement("{CALL sp_vehicle_entry(?,?,?,?)}")
        SET IN params; REGISTER OUT param sessionId as INTEGER
        cs.execute() -> RETURN cs.getInt(4)

        FUNCTION recordExit(sessionId)
            cs <- CallableStatement("{CALL sp_vehicle_exit(?,?)}")
            SET IN param; REGISTER OUT param fee as DECIMAL
            cs.execute() -> RETURN cs.getBigDecimal(2)
        END recordExit
    END SessionDAO

```

7. Parking Pass Module

```

BEGIN PassDAO
    FUNCTION addPass(pass)
        sql <- "INSERT INTO parking_passes(user_id,vehicle_id,zone_id,pass_type,
start_date,end_date,pass_fee,status) VALUES(?,?,?,?,?,?,?, 'Active')"
        EXECUTE PreparedStatement -> RETURN generatedKey

    FUNCTION expireOldPasses()
        sql <- "UPDATE parking_passes SET status='Expired'
WHERE end_date < CURDATE() AND status='Active'"
        EXECUTE plain Statement (batch update)

    FUNCTION autoCalculateEndDate(startDate, passType)
        Monthly -> start + 1 month
        Semester -> start + 6 months
        Annual -> start + 1 year
    END autoCalculateEndDate
END PassDAO

```

8. Payment Module

```

BEGIN PaymentDAO
    FUNCTION addPayment(payment)
        sql <- "INSERT INTO
payments(user_id,session_id,pass_id,amount,payment_mode,status)

```

```

VALUES (?, ?, ?, ?, ?, ?) "
SET sessionId/passId as NULL if not applicable
EXECUTE PreparedStatement -> RETURN generatedKey

FUNCTION getAllPayments()
    sql <- "SELECT p.*, u.full_name FROM payments p JOIN users u ORDER BY
payment_date DESC"
    EXECUTE plain Statement -> RETURN payment list
END PaymentDAO

```

9. Violation Module

```

BEGIN ViolationDAO
    FUNCTION addViolation(violation)
        VALIDATE vehicle registered, fineAmount > 0
        sql <- "INSERT INTO violations(vehicle_id,slot_id,violation_type,
description,fine_amount,status) VALUES(?,?,?,?,?, 'Pending')"
        EXECUTE PreparedStatement -> RETURN success

    FUNCTION updateViolationStatus(violationId, status)
        sql <- "UPDATE violations SET status=? WHERE violation_id=?"
        EXECUTE PreparedStatement (status = 'Paid' or 'Waived')

    FUNCTION searchViolations(keyword)
        sql <- "SELECT * FROM violations JOIN vehicles WHERE license_plate LIKE ? OR
violation_type LIKE ?"
        EXECUTE PreparedStatement -> RETURN list
END ViolationDAO

```

10. Reports Module

```

BEGIN ReportsPanel
    FUNCTION zoneOccupancyReport()
        sql <- "SELECT zone_name, total_slots, available, occupied FROM parking_zones
JOIN parking_slots GROUP BY zone_id"
        EXECUTE plain Statement -> FILL table, SHOW occupancy%

    FUNCTION revenueReport()
        sql <- "SELECT zone_name, SUM(session_fee), SUM(pass_fee) FROM zones
JOIN sessions JOIN passes GROUP BY zone_id"
        EXECUTE plain Statement -> SHOW total revenue

    FUNCTION dailyRevenueReport()
        sql <- "SELECT DATE(payment_date), COUNT(*), SUM(amount) FROM payments
WHERE status='Paid' GROUP BY DATE ORDER BY date DESC LIMIT 30"
        EXECUTE plain Statement -> FILL last 30 days table

    FUNCTION utilizationReport()
        sql <- "SELECT zone_name, ROUND(occupied*100/total) AS pct FROM zones
JOIN slots GROUP BY zone_id ORDER BY pct DESC"
        EXECUTE plain Statement -> DISPLAY utilization
END ReportsPanel

```

11. Main Application Flow

```

BEGIN Application
    FUNCTION main()
        SET UIManager <- SystemLookAndFeel
        CREATE MainFrame -> SHOW

BEGIN MainFrame
    ON STARTUP
        CREATE 7 panels: UserVehicle, ZoneSlot, Reservation,
EntryExit, PassPayment, Violation, Reports
        BUILD JMenuBar: File | Manage | Reports | Database | Help

```

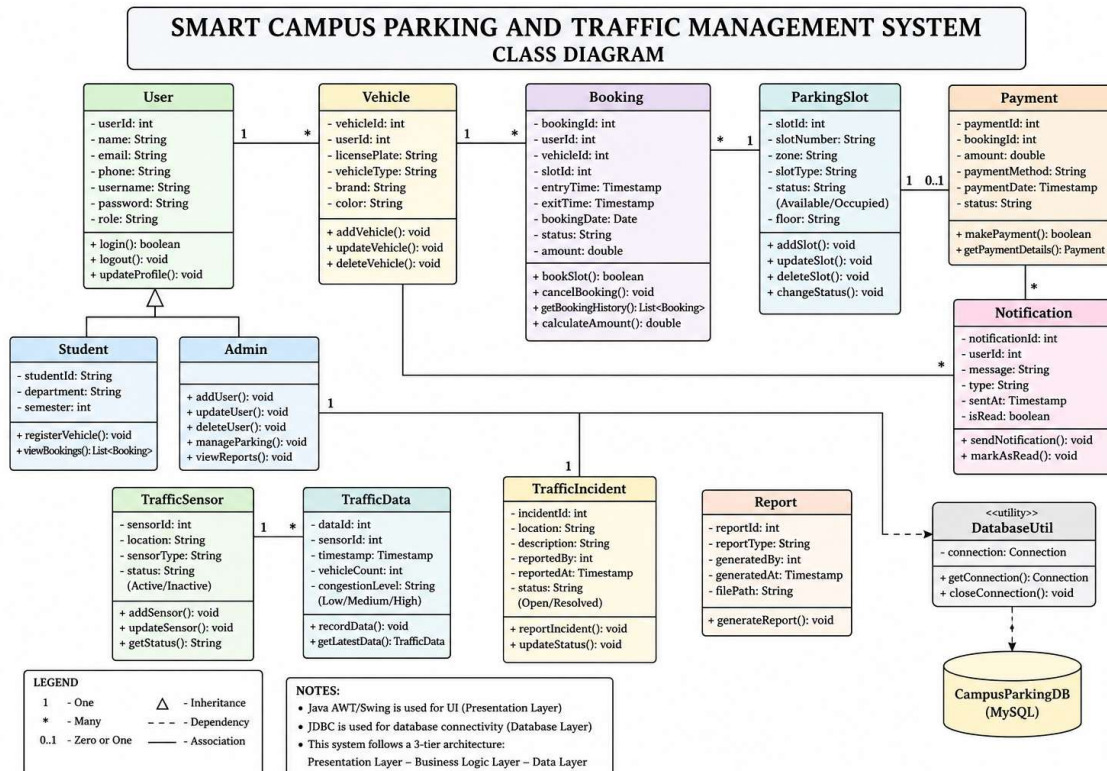
```

ADD header (dark blue), JTabbedPane (7 tabs), status bar
TEST DBConnection on startup
    IF success -> "Connected to campus_parking database"
    IF failure -> SHOW warning dialog with fix steps

ON MENU Refresh All -> CALL refresh() on all 7 panels
ON MENU Exit        -> confirm -> DBConnection.close() -> System.exit(0)
END MainFrame

```

PART 2 – CLASS DIAGRAM



PART 3 – SYSTEM DESIGN

Technology Stack

Component	Technology
Language	Java 21
GUI Framework	Java Swing – JFrame, JTabbedPane, JTable, GridBagLayout, JSplitPane, JMenuBar
Database	MySQL 8.0
Connectivity	JDBC – Statement, PreparedStatement, CallableStatement
Driver	mysql-connector-j-8.3.0.jar
Build	javac JDK 21 – 51 .class files

Database Schema – 9 Tables

```
users      (user_id PK, full_name, email UNIQUE, phone, user_type, id_number
UNIQUE, address)
vehicles   (vehicle_id PK, user_id FK->users, license_plate UNIQUE, vehicle_type,
make, model, color)
parking_zones (zone_id PK, zone_name UNIQUE, zone_type, location, total_slots,
hourly rate)
parking_slots (slot_id PK, zone_id FK->zones, slot_number, slot_type, status)
reservations (reservation_id PK, user_id FK, vehicle_id FK, slot_id FK,
reserved_from, reserved_until, status)
parking_sessions (session_id PK, vehicle_id FK, slot_id FK, user_id FK, entry_time,
exit_time, duration_hrs, fee_amount, status)
parking_passes (pass_id PK, user_id FK, vehicle_id FK, zone_id FK, pass_type,
start_date, end_date, pass_fee, status)
payments   (payment_id PK, user_id FK, session_id FK, pass_id FK, amount,
payment_mode, payment_date, status)
violations (violation_id PK, vehicle_id FK, slot_id FK, violation_type,
description, fine_amount, violation_date, status)
```

JDBC Statement Types Used

Type	Used In	Purpose
Statement	getAllUsers(), getAllZones(), Reports	Simple SELECT queries – no parameters
PreparedStatement	All INSERT, UPDATE, DELETE, filtered SELECT	Parameterized – prevents SQL injection
CallableStatement	SessionDAO.recordEntry/recordExit	Invokes stored procedures with IN/OUT params

Project Structure

```
parking/
├── database.sql          <- MySQL schema + stored procedures + sample data
├── lib/mysql-connector-j.jar <- JDBC driver
├── src/parking/
│   ├── db/DBConnection.java <- Singleton JDBC connection
│   └── model/ (9 entity classes) <- User, Vehicle, ParkingZone, ParkingSlot,
Payment, Violation
├── dao/ (9 DAO classes) <- UserDAO, VehicleDAO, ParkingZoneDAO,
ParkingSlotDAO,
```

```
| | |
PaymentDAO, ViolationDAO      ReservationDAO, SessionDAO, PassDAO,
| └─ ui/      (7 Swing classes)  <- MainFrame, UserVehiclePanel, ZoneSlotPanel,
PassPaymentPanel,             ReservationPanel, EntryExitPanel,
|                               ViolationPanel, ReportsPanel
└─ out/  (51 compiled .class files)
```


PART 4 – IMPLEMENTATION CODE

DBConnection.java

```
package parking.db;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * Singleton JDBC connection manager for campus_parking MySQL database.
 */
public class DBConnection {

    private static final String URL      =
"jdbc:mysql://localhost:3306/campus_parking?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true";
    private static final String USER     = "root";
    private static final String PASSWORD = "FORZA_FERRARI"; // MySQL password

    private static Connection connection = null;

    private DBConnection() {}

    public static Connection getConnection() throws SQLException {
        if (connection == null || connection.isClosed()) {
            try {
                Class.forName("com.mysql.cj.jdbc.Driver");
                connection = DriverManager.getConnection(URL, USER, PASSWORD);
            } catch (ClassNotFoundException e) {
                throw new SQLException("MySQL JDBC Driver not found. Add mysql-connector-j to /lib folder.\n" + e.getMessage());
            }
        }
        return connection;
    }

    public static void close() {
        try {
            if (connection != null && !connection.isClosed()) {
                connection.close();
                connection = null;
            }
        } catch (SQLException ignored) {}
    }

    /** Quick connectivity test – returns null on success, error message on failure.
    */
    public static String testConnection() {
        try {
            Connection c = getConnection();
            if (c != null && !c.isClosed()) return null;
            return "Connection returned null.";
        } catch (SQLException e) {
            return e.getMessage();
        }
    }
}
```

User.java

```
package parking.model;

import java.sql.Timestamp;

public class User {
    private int userId;
    private String fullName;
    private String email;
    private String phone;
    private String userType;    // Student, Faculty, Staff, Visitor, Service
    private String idNumber;
    private String address;
    private Timestamp createdAt;

    public User() {}

    public User(int userId, String fullName, String email, String phone,
                String userType, String idNumber, String address, Timestamp createdAt)
    {
        this.userId      = userId;
        this.fullName     = fullName;
        this.email        = email;
        this.phone        = phone;
        this.userType     = userType;
        this.idNumber     = idNumber;
        this.address      = address;
        this.createdAt    = createdAt;
    }

    // Getters & Setters
    public int      getUserId()      { return userId; }
    public void     setUserId(int v)  { userId = v; }
    public String   getFullName()     { return fullName; }
    public void     setFullName(String v) { fullName = v; }
    public String   getEmail()        { return email; }
    public void     setEmail(String v) { email = v; }
    public String   getPhone()        { return phone; }
    public void     setPhone(String v) { phone = v; }
    public String   getUserType()     { return userType; }
    public void     setUserType(String v) { userType = v; }
    public String   getIdNumber()     { return idNumber; }
    public void     setIdNumber(String v) { idNumber = v; }
    public String   getAddress()      { return address; }
    public void     setAddress(String v) { address = v; }
    public Timestamp getCreatedAt()    { return createdAt; }
    public void     setCreatedAt(Timestamp v) { createdAt = v; }

    @Override
    public String toString() { return userId + " - " + fullName + " (" + userType +
    ")"; }
}
```

Vehicle.java

```
package parking.model;

import java.sql.Timestamp;

public class Vehicle {
    private int vehicleId;
```

```

private int userId;
private String licensePlate;
private String vehicleType; // Car, Motorcycle, Truck, Bus, Bicycle, Other
private String make;
private String model;
private String color;
private Timestamp registeredAt;
// for display joins
private String ownerName;

public Vehicle() {}

public Vehicle(int vehicleId, int userId, String licensePlate, String vehicleType,
               String make, String model, String color, Timestamp registeredAt) {
    this.vehicleId = vehicleId;
    this.userId = userId;
    this.licensePlate = licensePlate;
    this.vehicleType = vehicleType;
    this.make = make;
    this.model = model;
    this.color = color;
    this.registeredAt = registeredAt;
}

public int getVehicleId() { return vehicleId; }
public void setVehicleId(int v) { vehicleId = v; }
public int getUserId() { return userId; }
public void setUserId(int v) { userId = v; }
public String getLicensePlate() { return licensePlate; }
public void setLicensePlate(String v) { licensePlate = v; }
public String getVehicleType() { return vehicleType; }
public void setVehicleType(String v) { vehicleType = v; }
public String getMake() { return make; }
public void setMake(String v) { make = v; }
public String getModel() { return model; }
public void setModel(String v) { model = v; }
public String getColor() { return color; }
public void setColor(String v) { color = v; }
public Timestamp getRegisteredAt() { return registeredAt; }
public void setRegisteredAt(Timestamp v) { registeredAt = v; }
public String getOwnerName() { return ownerName; }
public void setOwnerName(String v) { ownerName = v; }

@Override
public String toString() { return vehicleId + " - " + licensePlate + " (" +
vehicleType + ")"; }
}

```

ParkingZone.java

```

package parking.model;

import java.math.BigDecimal;

public class ParkingZone {
    private int zoneId;
    private String zoneName;
    private String zoneType; // Student, Faculty, Staff, Visitor, Service,
General
    private String location;
    private int totalSlots;
    private BigDecimal hourlyRate;
}

```

```

private String description;
// computed at runtime
private int availableSlots;

public ParkingZone() {}

public ParkingZone(int zoneId, String zoneName, String zoneType, String location,
int totalSlots, BigDecimal hourlyRate, String description) {
    this.zoneId = zoneId;
    this.zoneName = zoneName;
    this.zoneType = zoneType;
    this.location = location;
    this.totalSlots = totalSlots;
    this.hourlyRate = hourlyRate;
    this.description = description;
}

public int getZoneId() { return zoneId; }
public void setZoneId(int v) { zoneId = v; }
public String getZoneName() { return zoneName; }
public void setZoneName(String v) { zoneName = v; }
public String getZoneType() { return zoneType; }
public void setZoneType(String v) { zoneType = v; }
public String getLocation() { return location; }
public void setLocation(String v) { location = v; }
public int getTotalSlots() { return totalSlots; }
public void setTotalSlots(int v) { totalSlots = v; }
public BigDecimal getHourlyRate() { return hourlyRate; }
public void setHourlyRate(BigDecimal v) { hourlyRate = v; }
public String getDescription() { return description; }
public void setDescription(String v) { description = v; }
public int getAvailableSlots() { return availableSlots; }
public void setAvailableSlots(int v) { availableSlots = v; }

@Override
public String toString() { return zoneId + " - " + zoneName + " (" + zoneType +
");"; }
}

```

ParkingSlot.java

```

package parking.model;

public class ParkingSlot {
    private int slotId;
    private int zoneId;
    private String slotNumber;
    private String slotType; // Regular, Handicapped, EV, Reserved
    private String status; // Available, Occupied, Reserved, Maintenance
    // for display joins
    private String zoneName;

    public ParkingSlot() {}

    public ParkingSlot(int slotId, int zoneId, String slotNumber, String slotType,
String status) {
        this.slotId = slotId;
        this.zoneId = zoneId;
        this.slotNumber = slotNumber;
        this.slotType = slotType;
        this.status = status;
    }
}

```

```

    public int      getSlotId()                { return slotId; }
    public void     setSlotId(int v)           { slotId = v; }
    public int      getZoneId()                { return zoneId; }
    public void     setZoneId(int v)           { zoneId = v; }
    public String   getSlotNumber()            { return slotNumber; }
    public void     setSlotNumber(String v)    { slotNumber = v; }
    public String   getSlotType()              { return slotType; }
    public void     setSlotType(String v)      { slotType = v; }
    public String   getStatus()                { return status; }
    public void     setStatus(String v)        { status = v; }
    public String   getZoneName()              { return zoneName; }
    public void     setZoneName(String v)      { zoneName = v; }

    @Override
    public String toString() { return slotId + " - " + slotNumber + " [" + status +
    "]" + " "; }
}

```

Reservation.java

```

package parking.model;

import java.sql.Timestamp;

public class Reservation {
    private int reservationId;
    private int userId;
    private int vehicleId;
    private int slotId;
    private Timestamp reservedFrom;
    private Timestamp reservedUntil;
    private String status; // Active, Completed, Cancelled, Expired
    private Timestamp createdAt;
    // display joins
    private String userName;
    private String licensePlate;
    private String slotNumber;
    private String zoneName;

    public Reservation() {}

    public int      getReservationId()          { return reservationId; }
    public void     setReservationId(int v)      { reservationId = v; }
    public int      getUserId()                 { return userId; }
    public void     setUserId(int v)             { userId = v; }
    public int      getVehicleId()              { return vehicleId; }
    public void     setVehicleId(int v)          { vehicleId = v; }
    public int      getSlotId()                 { return slotId; }
    public void     setSlotId(int v)             { slotId = v; }
    public Timestamp getReservedFrom()           { return reservedFrom; }
    public void     setReservedFrom(Timestamp v) { reservedFrom = v; }
    public Timestamp getReservedUntil()          { return reservedUntil; }
    public void     setReservedUntil(Timestamp v) { reservedUntil = v; }
    public String   getStatus()                 { return status; }
    public void     setStatus(String v)          { status = v; }
    public Timestamp getCreatedAt()              { return createdAt; }
    public void     setCreatedAt(Timestamp v)    { createdAt = v; }
    public String   getUserName()               { return userName; }
    public void     setUserName(String v)        { userName = v; }
    public String   getLicensePlate()            { return licensePlate; }
    public void     setLicensePlate(String v)     { licensePlate = v; }
}

```

```

    public String      getSlotNumber()          { return slotNumber; }
    public void        setSlotNumber(String v)  { slotNumber = v; }
    public String      getZoneName()           { return zoneName; }
    public void        setZoneName(String v)    { zoneName = v; }
}

```

ParkingSession.java

```

package parking.model;

import java.math.BigDecimal;
import java.sql.Timestamp;

public class ParkingSession {
    private int sessionId;
    private int vehicleId;
    private int slotId;
    private int userId;
    private Timestamp entryTime;
    private Timestamp exitTime;
    private BigDecimal durationHrs;
    private BigDecimal feeAmount;
    private String status; // Active, Completed
    // display joins
    private String licensePlate;
    private String slotNumber;
    private String zoneName;
    private String userName;

    public ParkingSession() {}

    public int      getSessionId()          { return sessionId; }
    public void      setSessionId(int v)    { sessionId = v; }
    public int      getVehicleId()          { return vehicleId; }
    public void      setVehicleId(int v)    { vehicleId = v; }
    public int      getSlotId()             { return slotId; }
    public void      setSlotId(int v)       { slotId = v; }
    public int      getUserId()             { return userId; }
    public void      setUserId(int v)       { userId = v; }
    public Timestamp getEntryTime()          { return entryTime; }
    public void      setEntryTime(Timestamp v) { entryTime = v; }
    public Timestamp getExitTime()          { return exitTime; }
    public void      setExitTime(Timestamp v) { exitTime = v; }
    public BigDecimal getDurationHrs()       { return durationHrs; }
    public void      setDurationHrs(BigDecimal v) { durationHrs = v; }
    public BigDecimal getFeeAmount()        { return feeAmount; }
    public void      setFeeAmount(BigDecimal v) { feeAmount = v; }
    public String    getStatus()             { return status; }
    public void      setStatus(String v)     { status = v; }
    public String    getLicensePlate()       { return licensePlate; }
    public void      setLicensePlate(String v) { licensePlate = v; }
    public String    getSlotNumber()        { return slotNumber; }
    public void      setSlotNumber(String v) { slotNumber = v; }
    public String    getZoneName()          { return zoneName; }
    public void      setZoneName(String v)  { zoneName = v; }
    public String    getUserName()          { return userName; }
    public void      setUserName(String v)  { userName = v; }
}

```

ParkingPass.java

```
package parking.model;

import java.math.BigDecimal;
import java.sql.Date;
import java.sql.Timestamp;

public class ParkingPass {
    private int passId;
    private int userId;
    private int vehicleId;
    private int zoneId;
    private String passType; // Monthly, Semester, Annual
    private Date startDate;
    private Date endDate;
    private BigDecimal passFee;
    private String status; // Active, Expired, Cancelled
    private Timestamp issuedAt;
    // display joins
    private String userName;
    private String licensePlate;
    private String zoneName;

    public ParkingPass() {}

    public int getPassId() { return passId; }
    public void setPassId(int v) { passId = v; }
    public int getUserId() { return userId; }
    public void setUserId(int v) { userId = v; }
    public int getVehicleId() { return vehicleId; }
    public void setVehicleId(int v) { vehicleId = v; }
    public int getZoneId() { return zoneId; }
    public void setZoneId(int v) { zoneId = v; }
    public String getPassType() { return passType; }
    public void setPassType(String v) { passType = v; }
    public Date getStartDate() { return startDate; }
    public void setStartDate(Date v) { startDate = v; }
    public Date getEndDate() { return endDate; }
    public void setEndDate(Date v) { endDate = v; }
    public BigDecimal getPassFee() { return passFee; }
    public void setPassFee(BigDecimal v) { passFee = v; }
    public String getStatus() { return status; }
    public void setStatus(String v) { status = v; }
    public Timestamp getIssuedAt() { return issuedAt; }
    public void setIssuedAt(Timestamp v) { issuedAt = v; }
    public String getUserName() { return userName; }
    public void setUserName(String v) { userName = v; }
    public String getLicensePlate() { return licensePlate; }
    public void setLicensePlate(String v) { licensePlate = v; }
    public String getZoneName() { return zoneName; }
    public void setZoneName(String v) { zoneName = v; }
}
```

Payment.java

```
package parking.model;

import java.math.BigDecimal;
import java.sql.Timestamp;

public class Payment {
    private int paymentId;
```

```

private int userId;
private Integer sessionId;    // nullable
private Integer passId;      // nullable
private BigDecimal amount;
private String paymentMode;   // Cash, Card, UPI, Online
private Timestamp paymentDate;
private String status;        // Paid, Pending, Failed
// display joins
private String userName;

public Payment() {}

public int      getPaymentId()           { return paymentId; }
public void     setPaymentId(int v)       { paymentId = v; }
public int      getUserId()              { return userId; }
public void     setUserId(int v)          { userId = v; }
public Integer  getSessionId()           { return sessionId; }
public void     setSessionId(Integer v)   { sessionId = v; }
public Integer  getPassId()              { return passId; }
public void     setPassId(Integer v)      { passId = v; }
public BigDecimal getAmount()            { return amount; }
public void     setAmount(BigDecimal v)   { amount = v; }
public String   getPaymentMode()         { return paymentMode; }
public void     setPaymentMode(String v)  { paymentMode = v; }
public Timestamp getPaymentDate()        { return paymentDate; }
public void     setPaymentDate(Timestamp v) { paymentDate = v; }
public String   getStatus()              { return status; }
public void     setStatus(String v)       { status = v; }
public String   getUserName()            { return userName; }
public void     setUserName(String v)     { userName = v; }
}

```

Violation.java

```

package parking.model;

import java.math.BigDecimal;
import java.sql.Timestamp;

public class Violation {
    private int violationId;
    private int vehicleId;
    private Integer slotId;    // nullable
    private String violationType;
    private String description;
    private BigDecimal fineAmount;
    private Timestamp violationDate;
    private String status;    // Pending, Paid, Waived
    // display joins
    private String licensePlate;
    private String slotNumber;

    public Violation() {}

    public int      getViolationId()       { return violationId; }
    public void     setViolationId(int v)   { violationId = v; }
    public int      getVehicleId()         { return vehicleId; }
    public void     setVehicleId(int v)     { vehicleId = v; }
    public Integer  getSlotId()            { return slotId; }
    public void     setSlotId(Integer v)    { slotId = v; }
    public String   getViolationType()     { return violationType; }
    public void     setViolationType(String v) { violationType = v; }
}

```



```

    public String      getDescription()      { return description; }
    public void        setDescription(String v) { description = v; }
    public BigDecimal  getFineAmount()      { return fineAmount; }
    public void        setFineAmount(BigDecimal v) { fineAmount = v; }
    public Timestamp   getViolationDate()    { return violationDate; }
    public void        setViolationDate(Timestamp v) { violationDate = v; }
    public String      getStatus()          { return status; }
    public void        setStatus(String v)    { status = v; }
    public String      getLicensePlate()     { return licensePlate; }
    public void        setLicensePlate(String v) { licensePlate = v; }
    public String      getSlotNumber()       { return slotNumber; }
    public void        setSlotNumber(String v) { slotNumber = v; }
}

```

UserDAO.java

```

package parking.dao;

import parking.db.DBConnection;
import parking.model.User;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * UserDAO - uses PreparedStatement for all DML; Statement for simple reads.
 */
public class UserDAO {

    // --- INSERT ---
    public boolean addUser(User u) throws SQLException {
        String sql = "INSERT INTO users(full_name,email,phone,user_type,id_number,address) VALUES(?,?,?,?,?,?)";
        try (PreparedStatement ps = DBConnection.getConnection().prepareStatement(sql)) {
            ps.setString(1, u.getFullName());
            ps.setString(2, u.getEmail());
            ps.setString(3, u.getPhone());
            ps.setString(4, u.getUserType());
            ps.setString(5, u.getIdNumber());
            ps.setString(6, u.getAddress());
            return ps.executeUpdate() > 0;
        }
    }

    // --- UPDATE ---
    public boolean updateUser(User u) throws SQLException {
        String sql = "UPDATE users SET full_name=?,email=?,phone=?,user_type=?,id_number=?,address=? WHERE user_id=?";
        try (PreparedStatement ps = DBConnection.getConnection().prepareStatement(sql)) {
            ps.setString(1, u.getFullName());
            ps.setString(2, u.getEmail());
            ps.setString(3, u.getPhone());
            ps.setString(4, u.getUserType());
            ps.setString(5, u.getIdNumber());
            ps.setString(6, u.getAddress());
            ps.setInt(7, u.getUserId());
            return ps.executeUpdate() > 0;
        }
    }
}

```

```

// — DELETE —————
public boolean deleteUser(int userId) throws SQLException {
    String sql = "DELETE FROM users WHERE user_id=?";
    try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
        ps.setInt(1, userId);
        return ps.executeUpdate() > 0;
    }
}

// — SELECT ALL —————
public List<User> getAllUsers() throws SQLException {
    List<User> list = new ArrayList<>();
    // Using plain Statement (as required by rubric for SELECT variety)
    try (Statement st = DBConnection.getConnection().createStatement();
        ResultSet rs = st.executeQuery("SELECT * FROM users ORDER BY full_name"))
    {
        while (rs.next()) list.add(map(rs));
    }
    return list;
}

// — SELECT BY ID —————
public User getUserById(int userId) throws SQLException {
    String sql = "SELECT * FROM users WHERE user_id=?";
    try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
        ps.setInt(1, userId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return map(rs);
        }
    }
    return null;
}

// — SEARCH —————
public List<User> searchUsers(String keyword) throws SQLException {
    List<User> list = new ArrayList<>();
    String sql = "SELECT * FROM users WHERE full_name LIKE ? OR email LIKE ? OR
id_number LIKE ? OR phone LIKE ?";
    try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
        String k = "%" + keyword + "%";
        ps.setString(1, k); ps.setString(2, k);
        ps.setString(3, k); ps.setString(4, k);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(map(rs));
        }
    }
    return list;
}

// — FILTER BY TYPE —————
public List<User> getUsersByType(String userType) throws SQLException {
    List<User> list = new ArrayList<>();
    String sql = "SELECT * FROM users WHERE user_type=? ORDER BY full_name";
    try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
        ps.setString(1, userType);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(map(rs));
        }
    }
    return list;
}

```

```

    }

    // — MAP ROW —————
    private User map(ResultSet rs) throws SQLException {
        User u = new User();
        u.setUserId(rs.getInt("user_id"));
        u.setFullName(rs.getString("full_name"));
        u.setEmail(rs.getString("email"));
        u.setPhone(rs.getString("phone"));
        u.setUserType(rs.getString("user_type"));
        u.setIdNumber(rs.getString("id_number"));
        u.setAddress(rs.getString("address"));
        u.setCreatedAt(rs.getTimestamp("created_at"));
        return u;
    }
}

```

VehicleDAO.java

```

package parking.dao;

import parking.db.DBConnection;
import parking.model.Vehicle;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class VehicleDAO {

    public boolean addVehicle(Vehicle v) throws SQLException {
        String sql = "INSERT INTO
vehicles(user_id,license_plate,vehicle_type,make,model,color) VALUES(?,?,?,?,?,?)";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, v.getUserId());
            ps.setString(2, v.getLicensePlate());
            ps.setString(3, v.getVehicleType());
            ps.setString(4, v.getMake());
            ps.setString(5, v.getModel());
            ps.setString(6, v.getColor());
            return ps.executeUpdate() > 0;
        }
    }

    public boolean updateVehicle(Vehicle v) throws SQLException {
        String sql = "UPDATE vehicles SET
user_id=?,license_plate=?,vehicle_type=?,make=?,model=?,color=? WHERE vehicle_id=?";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, v.getUserId());
            ps.setString(2, v.getLicensePlate());
            ps.setString(3, v.getVehicleType());
            ps.setString(4, v.getMake());
            ps.setString(5, v.getModel());
            ps.setString(6, v.getColor());
            ps.setInt(7, v.getVehicleId());
            return ps.executeUpdate() > 0;
        }
    }

    public boolean deleteVehicle(int vehicleId) throws SQLException {

```

```

        String sql = "DELETE FROM vehicles WHERE vehicle_id=?";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, vehicleId);
            return ps.executeUpdate() > 0;
        }
    }

    public List<Vehicle> getAllVehicles() throws SQLException {
        List<Vehicle> list = new ArrayList<>();
        String sql = "SELECT v.*, u.full_name AS owner_name FROM vehicles v JOIN users
u ON v.user_id=u.user_id ORDER BY v.license_plate";
        try (Statement st = DBConnection.getConnection().createStatement();
            ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) list.add(map(rs));
        }
        return list;
    }

    public Vehicle getVehicleById(int vehicleId) throws SQLException {
        String sql = "SELECT v.*, u.full_name AS owner_name FROM vehicles v JOIN users
u ON v.user_id=u.user_id WHERE v.vehicle_id=?";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, vehicleId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return map(rs);
            }
        }
        return null;
    }

    public Vehicle getVehicleByPlate(String plate) throws SQLException {
        String sql = "SELECT v.*, u.full_name AS owner_name FROM vehicles v JOIN users
u ON v.user_id=u.user_id WHERE v.license_plate=?";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setString(1, plate);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return map(rs);
            }
        }
        return null;
    }

    public List<Vehicle> getVehiclesByUser(int userId) throws SQLException {
        List<Vehicle> list = new ArrayList<>();
        String sql = "SELECT v.*, u.full_name AS owner_name FROM vehicles v JOIN users
u ON v.user_id=u.user_id WHERE v.user_id=?";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, userId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        }
        return list;
    }

    public List<Vehicle> searchVehicles(String keyword) throws SQLException {
        List<Vehicle> list = new ArrayList<>();
        String sql = "SELECT v.*, u.full_name AS owner_name FROM vehicles v JOIN users
u ON v.user_id=u.user_id WHERE v.license_plate LIKE ? OR v.make LIKE ? OR v.model LIKE
? OR v.color LIKE ?";
    }

```

```

        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            String k = "%" + keyword + "%";
            ps.setString(1, k); ps.setString(2, k);
            ps.setString(3, k); ps.setString(4, k);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        }
        return list;
    }

    private Vehicle map(ResultSet rs) throws SQLException {
        Vehicle v = new Vehicle();
        v.setVehicleId(rs.getInt("vehicle_id"));
        v.setUserId(rs.getInt("user_id"));
        v.setLicensePlate(rs.getString("license_plate"));
        v.setVehicleType(rs.getString("vehicle_type"));
        v.setMake(rs.getString("make"));
        v.setModel(rs.getString("model"));
        v.setColor(rs.getString("color"));
        v.setRegisteredAt(rs.getTimestamp("registered_at"));
        try { v.setOwnerName(rs.getString("owner_name")); } catch (SQLException
ignored) {}
        return v;
    }
}

```

ParkingZoneDAO.java

```

package parking.dao;

import parking.db.DBConnection;
import parking.model.ParkingZone;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ParkingZoneDAO {

    public boolean addZone(ParkingZone z) throws SQLException {
        String sql = "INSERT INTO
parking_zones(zone_name,zone_type,location,total_slots,hourly_rate,description)
VALUES(?,?,?,?,?,?)";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setString(1, z.getZoneName());
            ps.setString(2, z.getZoneType());
            ps.setString(3, z.getLocation());
            ps.setInt(4, z.getTotalSlots());
            ps.setBigDecimal(5, z.getHourlyRate());
            ps.setString(6, z.getDescription());
            return ps.executeUpdate() > 0;
        }
    }

    public boolean updateZone(ParkingZone z) throws SQLException {
        String sql = "UPDATE parking_zones SET
zone_name=?,zone_type=?,location=?,total_slots=?,hourly_rate=?,description=? WHERE
zone_id=?";
    }
}

```

```

        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setString(1, z.getZoneName());
            ps.setString(2, z.getZoneType());
            ps.setString(3, z.getLocation());
            ps.setInt(4, z.getTotalSlots());
            ps.setBigDecimal(5, z.getHourlyRate());
            ps.setString(6, z.getDescription());
            ps.setInt(7, z.getZoneId());
            return ps.executeUpdate() > 0;
        }
    }

    public boolean deleteZone(int zoneId) throws SQLException {
        String sql = "DELETE FROM parking_zones WHERE zone_id=?";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, zoneId);
            return ps.executeUpdate() > 0;
        }
    }

    public List<ParkingZone> getAllZones() throws SQLException {
        List<ParkingZone> list = new ArrayList<>();
        String sql = "SELECT z.*, (SELECT COUNT(*) FROM parking_slots s WHERE
s.zone_id=z.zone_id AND s.status='Available') AS available_slots FROM parking_zones z
ORDER BY z.zone_name";
        try (Statement st = DBConnection.getConnection().createStatement();
            ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) list.add(map(rs, true));
        }
        return list;
    }

    public ParkingZone getZoneById(int zoneId) throws SQLException {
        String sql = "SELECT z.*, (SELECT COUNT(*) FROM parking_slots s WHERE
s.zone_id=z.zone_id AND s.status='Available') AS available_slots FROM parking_zones z
WHERE z.zone_id=?";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, zoneId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return map(rs, true);
            }
        }
        return null;
    }

    private ParkingZone map(ResultSet rs, boolean withAvailable) throws SQLException {
        ParkingZone z = new ParkingZone();
        z.setZoneId(rs.getInt("zone_id"));
        z.setZoneName(rs.getString("zone_name"));
        z.setZoneType(rs.getString("zone_type"));
        z.setLocation(rs.getString("location"));
        z.setTotalSlots(rs.getInt("total_slots"));
        z.setHourlyRate(rs.getBigDecimal("hourly_rate"));
        z.setDescription(rs.getString("description"));
        if (withAvailable) {
            try { z.setAvailableSlots(rs.getInt("available_slots")); } catch
(SQLException ignored) {}
        }
        return z;
    }
}

```

ParkingSlotDAO.java

```
package parking.dao;

import parking.db.DBConnection;
import parking.model.ParkingSlot;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ParkingSlotDAO {

    public boolean addSlot(ParkingSlot s) throws SQLException {
        String sql = "INSERT INTO parking_slots(zone_id,slot_number,slot_type,status) VALUES(?,?,?,?)";
        try (PreparedStatement ps =
            DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, s.getZoneId());
            ps.setString(2, s.getSlotNumber());
            ps.setString(3, s.getSlotType());
            ps.setString(4, s.getStatus());
            return ps.executeUpdate() > 0;
        }
    }

    public boolean updateSlot(ParkingSlot s) throws SQLException {
        String sql = "UPDATE parking_slots SET zone_id=?,slot_number=?,slot_type=?,status=? WHERE slot_id=?";
        try (PreparedStatement ps =
            DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, s.getZoneId());
            ps.setString(2, s.getSlotNumber());
            ps.setString(3, s.getSlotType());
            ps.setString(4, s.getStatus());
            ps.setInt(5, s.getSlotId());
            return ps.executeUpdate() > 0;
        }
    }

    public boolean updateSlotStatus(int slotId, String status) throws SQLException {
        String sql = "UPDATE parking_slots SET status=? WHERE slot_id=?";
        try (PreparedStatement ps =
            DBConnection.getConnection().prepareStatement(sql)) {
            ps.setString(1, status);
            ps.setInt(2, slotId);
            return ps.executeUpdate() > 0;
        }
    }

    public boolean deleteSlot(int slotId) throws SQLException {
        String sql = "DELETE FROM parking_slots WHERE slot_id=?";
        try (PreparedStatement ps =
            DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, slotId);
            return ps.executeUpdate() > 0;
        }
    }

    public List<ParkingSlot> getAllSlots() throws SQLException {
        List<ParkingSlot> list = new ArrayList<>();
```

```

        String sql = "SELECT s.*, z.zone_name FROM parking_slots s JOIN parking_zones
z ON s.zone_id=z.zone_id ORDER BY z.zone_name, s.slot_number";
        try (Statement st = DBConnection.getConnection().createStatement();
            ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) list.add(map(rs));
        }
        return list;
    }

    public List<ParkingSlot> getSlotsByZone(int zoneId) throws SQLException {
        List<ParkingSlot> list = new ArrayList<>();
        String sql = "SELECT s.*, z.zone_name FROM parking_slots s JOIN parking_zones
z ON s.zone_id=z.zone_id WHERE s.zone_id=? ORDER BY s.slot_number";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, zoneId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        }
        return list;
    }

    public List<ParkingSlot> getAvailableSlots(int zoneId) throws SQLException {
        List<ParkingSlot> list = new ArrayList<>();
        String sql = "SELECT s.*, z.zone_name FROM parking_slots s JOIN parking_zones
z ON s.zone_id=z.zone_id WHERE s.zone_id=? AND s.status='Available' ORDER BY
s.slot_number";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, zoneId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        }
        return list;
    }

    public List<ParkingSlot> getSlotsByStatus(String status) throws SQLException {
        List<ParkingSlot> list = new ArrayList<>();
        String sql = "SELECT s.*, z.zone_name FROM parking_slots s JOIN parking_zones
z ON s.zone_id=z.zone_id WHERE s.status=? ORDER BY z.zone_name, s.slot_number";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setString(1, status);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        }
        return list;
    }

    public ParkingSlot getSlotById(int slotId) throws SQLException {
        String sql = "SELECT s.*, z.zone_name FROM parking_slots s JOIN parking_zones
z ON s.zone_id=z.zone_id WHERE s.slot_id=?";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, slotId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return map(rs);
            }
        }
        return null;
    }
}

```



```

        private ParkingSlot map(ResultSet rs) throws SQLException {
            ParkingSlot s = new ParkingSlot();
            s.setSlotId(rs.getInt("slot_id"));
            s.setZoneId(rs.getInt("zone_id"));
            s.setSlotNumber(rs.getString("slot_number"));
            s.setSlotType(rs.getString("slot_type"));
            s.setStatus(rs.getString("status"));
            try { s.setZoneName(rs.getString("zone_name")); } catch (SQLException ignored) {}
        }

        return s;
    }
}

```

ReservationDAO.java

```

package parking.dao;

import parking.db.DBConnection;
import parking.model.Reservation;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ReservationDAO {

    private static final String JOIN_SQL =
        "SELECT r.*, u.full_name AS user_name, v.license_plate, s.slot_number, " +
        "z.zone_name " +
        "FROM reservations r " +
        "JOIN users u ON r.user_id=u.user_id " +
        "JOIN vehicles v ON r.vehicle_id=v.vehicle_id " +
        "JOIN parking_slots s ON r.slot_id=s.slot_id " +
        "JOIN parking_zones z ON s.zone_id=z.zone_id ";

    public int addReservation(Reservation r) throws SQLException {
        // Check for conflicting active reservation on the same slot
        String checkSql = "SELECT COUNT(*) FROM reservations WHERE slot_id=? AND " +
            "status='Active' AND NOT (reserved_until<=? OR reserved_from>=?)";
        try (PreparedStatement chk =
            DBConnection.getConnection().prepareStatement(checkSql)) {
            chk.setInt(1, r.getSlotId());
            chk.setTimestamp(2, r.getReservedFrom());
            chk.setTimestamp(3, r.getReservedUntil());
            try (ResultSet rs = chk.executeQuery()) {
                if (rs.next() && rs.getInt(1) > 0)
                    throw new SQLException("Conflicting reservation exists for this " +
                        "slot in the chosen time window.");
            }
        }

        String sql = "INSERT INTO " +
            "reservations(user_id,vehicle_id,slot_id,reserved_from,reserved_until,status) " +
            "VALUES(?,?,?,?,?, 'Active')";
        try (PreparedStatement ps = DBConnection.getConnection().prepareStatement(sql,
            Statement.RETURN_GENERATED_KEYS)) {
            ps.setInt(1, r.getUserId());
            ps.setInt(2, r.getVehicleId());
            ps.setInt(3, r.getSlotId());
            ps.setTimestamp(4, r.getReservedFrom());
            ps.setTimestamp(5, r.getReservedUntil());
            ps.executeUpdate();
            // Mark slot as Reserved
            new ParkingSlotDAO().updateSlotStatus(r.getSlotId(), "Reserved");
        }
    }
}

```

```

        try (ResultSet gk = ps.getGeneratedKeys()) {
            if (gk.next()) return gk.getInt(1);
        }
    }
    return -1;
}

public boolean cancelReservation(int reservationId) throws SQLException {
    // fetch slot first
    Reservation r = getReservationById(reservationId);
    String sql = "UPDATE reservations SET status='Cancelled' WHERE reservation_id=?";
    try (PreparedStatement ps =
        DBConnection.getConnection().prepareStatement(sql)) {
        ps.setInt(1, reservationId);
        boolean ok = ps.executeUpdate() > 0;
        if (ok && r != null)
            new ParkingSlotDAO().updateSlotStatus(r.getSlotId(), "Available");
        return ok;
    }
}

public boolean updateStatus(int reservationId, String status) throws SQLException
{
    String sql = "UPDATE reservations SET status=? WHERE reservation_id=?";
    try (PreparedStatement ps =
        DBConnection.getConnection().prepareStatement(sql)) {
        ps.setString(1, status);
        ps.setInt(2, reservationId);
        return ps.executeUpdate() > 0;
    }
}

public List<Reservation> getAllReservations() throws SQLException {
    List<Reservation> list = new ArrayList<>();
    try (Statement st = DBConnection.getConnection().createStatement();
        ResultSet rs = st.executeQuery(JOIN_SQL + "ORDER BY r.created_at DESC"))
    {
        while (rs.next()) list.add(map(rs));
    }
    return list;
}

public Reservation getReservationById(int id) throws SQLException {
    try (PreparedStatement ps =
        DBConnection.getConnection().prepareStatement(JOIN_SQL + "WHERE r.reservation_id=?"))
    {
        ps.setInt(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return map(rs);
        }
    }
    return null;
}

public List<Reservation> getActiveReservationsBySlot(int slotId) throws
SQLException {
    List<Reservation> list = new ArrayList<>();
    try (PreparedStatement ps =
        DBConnection.getConnection().prepareStatement(JOIN_SQL + "WHERE r.slot_id=? AND
r.status='Active'")) {
        ps.setInt(1, slotId);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(map(rs));
        }
    }
}

```

```

    }
    return list;
}

private Reservation map(ResultSet rs) throws SQLException {
    Reservation r = new Reservation();
    r.setReservationId(rs.getInt("reservation_id"));
    r.setUserId(rs.getInt("user_id"));
    r.setVehicleId(rs.getInt("vehicle_id"));
    r.setSlotId(rs.getInt("slot_id"));
    r.setReservedFrom(rs.getTimestamp("reserved_from"));
    r.setReservedUntil(rs.getTimestamp("reserved_until"));
    r.setStatus(rs.getString("status"));
    r.setCreatedAt(rs.getTimestamp("created_at"));
    try { r.setUserName(rs.getString("user_name")); } catch (SQLException
ignored) {}
    try { r.setLicensePlate(rs.getString("license_plate")); } catch (SQLException
ignored) {}
    try { r.setSlotNumber(rs.getString("slot_number")); } catch (SQLException
ignored) {}
    try { r.setZoneName(rs.getString("zone_name")); } catch (SQLException
ignored) {}
    return r;
}
}

```

SessionDAO.java

```

package parking.dao;

import parking.db.DBConnection;
import parking.model.ParkingSession;

import java.math.BigDecimal;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * SessionDAO - uses CallableStatement for entry/exit stored procedures
 * and PreparedStatement for queries.
 */
public class SessionDAO {

    private static final String JOIN_SQL =
        "SELECT ps.*, v.license_plate, sl.slot_number, z.zone_name, u.full_name AS\n" +
        "user_name " +
        "FROM parking_sessions ps " +
        "JOIN vehicles v ON ps.vehicle_id=v.vehicle_id " +
        "JOIN parking_slots sl ON ps.slot_id=sl.slot_id " +
        "JOIN parking_zones z ON sl.zone_id=z.zone_id " +
        "JOIN users u ON ps.user_id=u.user_id ";

    // — ENTRY via CallableStatement —————
    public int recordEntry(int vehicleId, int slotId, int userId) throws SQLException
    {
        try (CallableStatement cs = DBConnection.getConnection().prepareCall("{CALL\n" +
        "sp_vehicle_entry(?,?,?,?)}")) {
            cs.setInt(1, vehicleId);
            cs.setInt(2, slotId);
            cs.setInt(3, userId);
            cs.registerOutParameter(4, Types.INTEGER);

```

```

        cs.execute();
        return cs.getInt(4);
    }
}

// — EXIT via CallableStatement —————
public BigDecimal recordExit(int sessionId) throws SQLException {
    try (CallableStatement cs = DBConnection.getConnection().prepareCall("{CALL
sp_vehicle_exit(?,?)}") {
        cs.setInt(1, sessionId);
        cs.registerOutParameter(2, Types.DECIMAL);
        cs.execute();
        return cs.getBigDecimal(2);
    }
}

// — Active session by license plate —————
public ParkingSession getActiveSessionByPlate(String plate) throws SQLException {
    String sql = JOIN_SQL + "WHERE v.license_plate=? AND ps.status='Active'";
    try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
        ps.setString(1, plate);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return map(rs);
        }
    }
    return null;
}

public ParkingSession getSessionById(int sessionId) throws SQLException {
    try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(JOIN_SQL + "WHERE ps.session_id=?")) {
        ps.setInt(1, sessionId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return map(rs);
        }
    }
    return null;
}

public List<ParkingSession> getActiveSessions() throws SQLException {
    List<ParkingSession> list = new ArrayList<>();
    try (Statement st = DBConnection.getConnection().createStatement();
        ResultSet rs = st.executeQuery(JOIN_SQL + "WHERE ps.status='Active' ORDER
BY ps.entry_time")) {
        while (rs.next()) list.add(map(rs));
    }
    return list;
}

public List<ParkingSession> getAllSessions() throws SQLException {
    List<ParkingSession> list = new ArrayList<>();
    try (Statement st = DBConnection.getConnection().createStatement();
        ResultSet rs = st.executeQuery(JOIN_SQL + "ORDER BY ps.entry_time DESC"))
    {
        while (rs.next()) list.add(map(rs));
    }
    return list;
}

public List<ParkingSession> getSessionsByVehicle(int vehicleId) throws
SQLException {
    List<ParkingSession> list = new ArrayList<>();

```

```

        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(JOIN_SQL + "WHERE ps.vehicle_id=? ORDER
BY ps.entry_time DESC")) {
            ps.setInt(1, vehicleId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        }
        return list;
    }

    private ParkingSession map(ResultSet rs) throws SQLException {
        ParkingSession s = new ParkingSession();
        s.setSessionId(rs.getInt("session_id"));
        s.setVehicleId(rs.getInt("vehicle_id"));
        s.setSlotId(rs.getInt("slot_id"));
        s.setUserId(rs.getInt("user_id"));
        s.setEntryTime(rs.getTimestamp("entry_time"));
        s.setExitTime(rs.getTimestamp("exit_time"));
        s.setDurationHrs(rs.getBigDecimal("duration_hrs"));
        s.setFeeAmount(rs.getBigDecimal("fee_amount"));
        s.setStatus(rs.getString("status"));
        try { s.setLicensePlate(rs.getString("license_plate")); } catch (SQLException
ignored) {}
        try { s.setSlotNumber(rs.getString("slot_number")); } catch (SQLException
ignored) {}
        try { s.setZoneName(rs.getString("zone_name")); } catch (SQLException
ignored) {}
        try { s.setUserName(rs.getString("user_name")); } catch (SQLException
ignored) {}
        return s;
    }
}

```

PassDAO.java

```

package parking.dao;

import parking.db.DBConnection;
import parking.model.ParkingPass;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class PassDAO {

    private static final String JOIN_SQL =
        "SELECT pp.*, u.full_name AS user_name, v.license_plate, z.zone_name " +
        "FROM parking_passes pp " +
        "JOIN users u ON pp.user_id=u.user_id " +
        "JOIN vehicles v ON pp.vehicle_id=v.vehicle_id " +
        "JOIN parking_zones z ON pp.zone_id=z.zone_id ";

    public int addPass(ParkingPass p) throws SQLException {
        String sql = "INSERT INTO
parking_passes(user_id,vehicle_id,zone_id,pass_type,start_date,end_date,pass_fee,statu
s) VALUES(?,?,?,?,?,?,?, 'Active')";
        try (PreparedStatement ps = DBConnection.getConnection().prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
            ps.setInt(1, p.getUserId());
            ps.setInt(2, p.getVehicleId());

```

```

        ps.setInt(3, p.getZoneId());
        ps.setString(4, p.getPassType());
        ps.setDate(5, p.getStartDate());
        ps.setDate(6, p.getEndDate());
        ps.setBigDecimal(7, p.getPassFee());
        ps.executeUpdate();
        try (ResultSet gk = ps.getGeneratedKeys()) {
            if (gk.next()) return gk.getInt(1);
        }
    }
    return -1;
}

public boolean updatePassStatus(int passId, String status) throws SQLException {
    String sql = "UPDATE parking_passes SET status=? WHERE pass_id=?";
    try (PreparedStatement ps =
        DBConnection.getConnection().prepareStatement(sql)) {
        ps.setString(1, status);
        ps.setInt(2, passId);
        return ps.executeUpdate() > 0;
    }
}

public boolean deletePass(int passId) throws SQLException {
    String sql = "DELETE FROM parking_passes WHERE pass_id=?";
    try (PreparedStatement ps =
        DBConnection.getConnection().prepareStatement(sql)) {
        ps.setInt(1, passId);
        return ps.executeUpdate() > 0;
    }
}

public List<ParkingPass> getAllPasses() throws SQLException {
    List<ParkingPass> list = new ArrayList<>();
    try (Statement st = DBConnection.getConnection().createStatement();
        ResultSet rs = st.executeQuery(JOIN_SQL + "ORDER BY pp.issued_at DESC"))
    {
        while (rs.next()) list.add(map(rs));
    }
    return list;
}

public ParkingPass getPassById(int passId) throws SQLException {
    try (PreparedStatement ps =
        DBConnection.getConnection().prepareStatement(JOIN_SQL + "WHERE pp.pass_id=?")) {
        ps.setInt(1, passId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return map(rs);
        }
    }
    return null;
}

public List<ParkingPass> getPassesByUser(int userId) throws SQLException {
    List<ParkingPass> list = new ArrayList<>();
    try (PreparedStatement ps =
        DBConnection.getConnection().prepareStatement(JOIN_SQL + "WHERE pp.user_id=? ORDER BY
        pp.issued_at DESC")) {
        ps.setInt(1, userId);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(map(rs));
        }
    }
    return list;
}

```

```

    /** Expire passes whose end_date < today */
    public void expireOldPasses() throws SQLException {
        String sql = "UPDATE parking_passes SET status='Expired' WHERE end_date <
CURDATE() AND status='Active'";
        try (Statement st = DBConnection.getConnection().createStatement()) {
            st.executeUpdate(sql);
        }
    }

    private ParkingPass map(ResultSet rs) throws SQLException {
        ParkingPass p = new ParkingPass();
        p.setPassId(rs.getInt("pass_id"));
        p.setUserId(rs.getInt("user_id"));
        p.setVehicleId(rs.getInt("vehicle_id"));
        p.setZoneId(rs.getInt("zone_id"));
        p.setPassType(rs.getString("pass_type"));
        p.setStartDate(rs.getDate("start_date"));
        p.setEndDate(rs.getDate("end_date"));
        p.setPassFee(rs.getBigDecimal("pass_fee"));
        p.setStatus(rs.getString("status"));
        p.setIssuedAt(rs.getTimestamp("issued_at"));
        try { p.setUserName(rs.getString("user_name")); } catch (SQLException
ignored) {}
        try { p.setLicensePlate(rs.getString("license_plate")); } catch (SQLException
ignored) {}
        try { p.setZoneName(rs.getString("zone_name")); } catch (SQLException
ignored) {}
        return p;
    }
}

```

PaymentDAO.java

```

package parking.dao;

import parking.db.DBConnection;
import parking.model.Payment;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class PaymentDAO {

    private static final String JOIN_SQL =
        "SELECT p.*, u.full_name AS user_name FROM payments p JOIN users u ON
p.user_id=u.user_id ";

    public int addPayment(Payment p) throws SQLException {
        String sql = "INSERT INTO
payments(user_id,session_id,pass_id,amount,payment_mode,status) VALUES(?,?,?, ?,?,?)";
        try (PreparedStatement ps = DBConnection.getConnection().prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
            ps.setInt(1, p.getUserId());
            if (p.getSessionId() != null) ps.setInt(2, p.getSessionId()); else
ps.setNull(2, Types.INTEGER);
            if (p.getPassId() != null) ps.setInt(3, p.getPassId()); else
ps.setNull(3, Types.INTEGER);
            ps.setBigDecimal(4, p.getAmount());
            ps.setString(5, p.getPaymentMode());
            ps.setString(6, p.getStatus() != null ? p.getStatus() : "Paid");

```

```

        ps.executeUpdate();
        try (ResultSet gk = ps.getGeneratedKeys()) {
            if (gk.next()) return gk.getInt(1);
        }
    }
    return -1;
}

public boolean updatePaymentStatus(int paymentId, String status) throws
SQLException {
    String sql = "UPDATE payments SET status=? WHERE payment_id=?";
    try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
        ps.setString(1, status);
        ps.setInt(2, paymentId);
        return ps.executeUpdate() > 0;
    }
}

public List<Payment> getAllPayments() throws SQLException {
    List<Payment> list = new ArrayList<>();
    try (Statement st = DBConnection.getConnection().createStatement();
        ResultSet rs = st.executeQuery(JOIN_SQL + "ORDER BY p.payment_date
DESC")) {
        while (rs.next()) list.add(map(rs));
    }
    return list;
}

public List<Payment> getPaymentsByUser(int userId) throws SQLException {
    List<Payment> list = new ArrayList<>();
    try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(JOIN_SQL + "WHERE p.user_id=? ORDER BY
p.payment_date DESC")) {
        ps.setInt(1, userId);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(map(rs));
        }
    }
    return list;
}

/** Total revenue grouped by date - used in reports */
public ResultSet getDailyRevenue() throws SQLException {
    String sql = "SELECT DATE(payment_date) AS pay_date, SUM(amount) AS total FROM
payments WHERE status='Paid' GROUP BY DATE(payment_date) ORDER BY pay_date DESC";
    return DBConnection.getConnection().createStatement().executeQuery(sql);
}

private Payment map(ResultSet rs) throws SQLException {
    Payment p = new Payment();
    p.setPaymentId(rs.getInt("payment_id"));
    p.setUserId(rs.getInt("user_id"));
    int sid = rs.getInt("session_id"); p.setSessionId(rs.isNull() ? null : sid);
    int pid = rs.getInt("pass_id"); p.setPassId(rs.isNull() ? null : pid);
    p.setAmount(rs.getBigDecimal("amount"));
    p.setPaymentMode(rs.getString("payment_mode"));
    p.setPaymentDate(rs.getTimestamp("payment_date"));
    p.setStatus(rs.getString("status"));
    try { p.setUserName(rs.getString("user_name")); } catch (SQLException ignored)
{}
    return p;
}
}

```


ViolationDAO.java

```
package parking.dao;

import parking.db.DBConnection;
import parking.model.Violation;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ViolationDAO {

    private static final String JOIN_SQL =
        "SELECT vl.*, v.license_plate, s.slot_number " +
        "FROM violations vl " +
        "JOIN vehicles v ON vl.vehicle_id=v.vehicle_id " +
        "LEFT JOIN parking_slots s ON vl.slot_id=s.slot_id ";

    public boolean addViolation(Violation viol) throws SQLException {
        String sql = "INSERT INTO
violations(vehicle_id,slot_id,violation_type,description,fine_amount,status)
VALUES(?,?,?,?,'Pending')";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, viol.getVehicleId());
            if (viol.getSlotId() != null) ps.setInt(2, viol.getSlotId()); else
ps.setNull(2, Types.INTEGER);
            ps.setString(3, viol.getViolationType());
            ps.setString(4, viol.getDescription());
            ps.setBigDecimal(5, viol.getFineAmount());
            return ps.executeUpdate() > 0;
        }
    }

    public boolean updateViolationStatus(int violationId, String status) throws
SQLException {
        String sql = "UPDATE violations SET status=? WHERE violation_id=?";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setString(1, status);
            ps.setInt(2, violationId);
            return ps.executeUpdate() > 0;
        }
    }

    public boolean deleteViolation(int violationId) throws SQLException {
        String sql = "DELETE FROM violations WHERE violation_id=?";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            ps.setInt(1, violationId);
            return ps.executeUpdate() > 0;
        }
    }

    public List<Violation> getAllViolations() throws SQLException {
        List<Violation> list = new ArrayList<>();
        try (Statement st = DBConnection.getConnection().createStatement();
            ResultSet rs = st.executeQuery(JOIN_SQL + "ORDER BY vl.violation_date
DESC")) {
            while (rs.next()) list.add(map(rs));
        }
    }
}
```

```

        return list;
    }

    public List<Violation> getPendingViolations() throws SQLException {
        List<Violation> list = new ArrayList<>();
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(JOIN_SQL + "WHERE vl.status='Pending'
ORDER BY vl.violation_date DESC")) {
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        }
        return list;
    }

    public List<Violation> getViolationsByVehicle(int vehicleId) throws SQLException {
        List<Violation> list = new ArrayList<>();
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(JOIN_SQL + "WHERE vl.vehicle_id=? ORDER
BY vl.violation_date DESC")) {
            ps.setInt(1, vehicleId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        }
        return list;
    }

    public List<Violation> searchViolations(String keyword) throws SQLException {
        List<Violation> list = new ArrayList<>();
        String sql = JOIN_SQL + "WHERE v.license_plate LIKE ? OR vl.violation_type
LIKE ? ORDER BY vl.violation_date DESC";
        try (PreparedStatement ps =
DBConnection.getConnection().prepareStatement(sql)) {
            String k = "%" + keyword + "%";
            ps.setString(1, k); ps.setString(2, k);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        }
        return list;
    }

    private Violation map(ResultSet rs) throws SQLException {
        Violation v = new Violation();
        v.setViolationId(rs.getInt("violation_id"));
        v.setVehicleId(rs.getInt("vehicle_id"));
        int sid = rs.getInt("slot_id"); v.setSlotId(rs.isNull() ? null : sid);
        v.setViolationType(rs.getString("violation_type"));
        v.setDescription(rs.getString("description"));
        v.setFineAmount(rs.getBigDecimal("fine_amount"));
        v.setViolationDate(rs.getTimestamp("violation_date"));
        v.setStatus(rs.getString("status"));
        try { v.setLicensePlate(rs.getString("license_plate")); } catch (SQLException
ignored) {}
        try { v.setSlotNumber(rs.getString("slot_number")); } catch (SQLException
ignored) {}
        return v;
    }
}

```

MainFrame.java

```
package parking.ui;

import parking.db.DBConnection;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

/**
 * MainFrame - root JFrame with a JTabbedPane and a full menu bar.
 * Demonstrates: JFrame, JMenuBar, JMenu, JMenuItem, JTabbedPane,
 * BorderLayout, event handling via ActionListener.
 */
public class MainFrame extends JFrame {

    private JTabbedPane tabs;
    private JLabel statusBar;

    // Panel instances (lazily created once)
    private UserVehiclePanel    userVehiclePanel;
    private ZoneSlotPanel       zoneSlotPanel;
    private ReservationPanel     reservationPanel;
    private EntryExitPanel      entryExitPanel;
    private PassPaymentPanel     passPaymentPanel;
    private ViolationPanel       violationPanel;
    private ReportsPanel         reportsPanel;

    public MainFrame() {
        super("Smart Campus Parking & Traffic Management System");
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        setSize(1200, 750);
        setMinimumSize(new Dimension(900, 600));
        setLocationRelativeTo(null);

        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { confirmExit(); }
        });

        initUI();
        testDBOnStartup();
    }

    // — UI SETUP —
    private void initUI() {
        setLayout(new BorderLayout());

        // — Header banner —
        JPanel header = new JPanel(new BorderLayout());
        header.setBackground(new Color(0, 70, 127));
        header.setPreferredSize(new Dimension(0, 55));
        JLabel title = new JLabel(" \uD83C\uDFDB Smart Campus Parking & Traffic Management System", JLabel.LEFT);
        title.setForeground(Color.WHITE);
        title.setFont(new Font("Segoe UI", Font.BOLD, 18));
        header.add(title, BorderLayout.WEST);
        JLabel dateLabel = new JLabel(new java.util.Date().toString() + " ", JLabel.RIGHT);
        dateLabel.setForeground(new Color(200, 230, 255));
        dateLabel.setFont(new Font("Segoe UI", Font.PLAIN, 11));
        header.add(dateLabel, BorderLayout.EAST);
        add(header, BorderLayout.NORTH);
    }
}
```

```

// — Tabbed pane —
tabs = new JTabbedPane(JTabbedPane.TOP);
tabs.setFont(new Font("Segoe UI", Font.PLAIN, 13));

userVehiclePanel = new UserVehiclePanel(this);
zoneSlotPanel    = new ZoneSlotPanel(this);
reservationPanel = new ReservationPanel(this);
entryExitPanel   = new EntryExitPanel(this);
passPaymentPanel = new PassPaymentPanel(this);
violationPanel   = new ViolationPanel(this);
reportsPanel     = new ReportsPanel(this);

tabs.addTab("\u00D3\u00C4 Users & Vehicles",    userVehiclePanel);
tabs.addTab("\u0083\u00D7 Zones & Slots",        zoneSlotPanel);
tabs.addTab("\u0083\u00D3 Reservations",         reservationPanel);
tabs.addTab("\u0083\u00E9 Entry / Exit",         entryExitPanel);
tabs.addTab("\u0083\u00DF Passes & Payments",    passPaymentPanel);
tabs.addTab("\u00A0 Violations",                violationPanel);
tabs.addTab("\u0083\u00C8 Reports",              reportsPanel);

add(tabs, BorderLayout.CENTER);

// — Status bar —
statusBar = new JLabel(" Ready");
statusBar.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createMatteBorder(1, 0, 0, 0, Color.GRAY),
    BorderFactory.createEmptyBorder(3, 6, 3, 6)));
statusBar.setFont(new Font("Segoe UI", Font.PLAIN, 12));
add(statusBar, BorderLayout.SOUTH);

// — Menu bar —
setJMenuBar(buildMenuBar());
}

// — MENU BAR —
private JMenuBar buildMenuBar() {
    JMenuBar mb = new JMenuBar();

    // File menu
    JMenu fileMenu = new JMenu("File");
    fileMenu.setMnemonic(KeyEvent.VK_F);
    JMenuItem miRefresh = new JMenuItem("Refresh All", KeyEvent.VK_R);
    miRefresh.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_F5, 0));
    miRefresh.addActionListener(e -> refreshAll());
    JMenuItem miExit = new JMenuItem("Exit", KeyEvent.VK_X);
    miExit.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_Q,
InputEvent.CTRL_DOWN_MASK));
    miExit.addActionListener(e -> confirmExit());
    fileMenu.add(miRefresh);
    fileMenu.addSeparator();
    fileMenu.add(miExit);

    // Manage menu
    JMenu manageMenu = new JMenu("Manage");
    String[] labels = {"Users & Vehicles", "Zones & Slots", "Reservations", "Entry
/ Exit", "Passes & Payments", "Violations"};
    for (int i = 0; i < labels.length; i++) {
        final int idx = i;
        JMenuItem mi = new JMenuItem(labels[i]);
        mi.addActionListener(e -> tabs.setSelectedIndex(idx));
        manageMenu.add(mi);
    }
}

```

```

// Reports menu
JMenu reportsMenu = new JMenu("Reports");
JMenuItem miReports = new JMenuItem("Open Reports");
miReports.addActionListener(e -> tabs.setSelectedIndex(6));
reportsMenu.add(miReports);

// Database menu
JMenu dbMenu = new JMenu("Database");
JMenuItem miTest = new JMenuItem("Test Connection");
miTest.addActionListener(e -> {
    String err = DBConnection.testConnection();
    if (err == null) showInfo("Database connection is OK.");
    else showError("Connection failed:\n" + err);
});
dbMenu.add(miTest);

// Help menu
JMenu helpMenu = new JMenu("Help");
JMenuItem miAbout = new JMenuItem("About");
miAbout.addActionListener(e -> JOptionPane.showMessageDialog(this,
    "Smart Campus Parking & Traffic Management System\n" +
    "Version 1.0 | CSA09 - Programming in Java\n" +
    "Built with Java Swing + JDBC (MySQL)\n\n" +
    "Covers: AWT/Swing, Event Handling, Layout Managers,\n" +
    "JDBC, PreparedStatement, CallableStatement, SQL CRUD",
    "About", JOptionPane.INFORMATION_MESSAGE));
helpMenu.add(miAbout);

mb.add(fileMenu);
mb.add(manageMenu);
mb.add(reportsMenu);
mb.add(dbMenu);
mb.add(helpMenu);
return mb;
}

// — HELPERS —————
public void setStatus(String msg) {
    statusBar.setText(" " + msg);
}

public void showError(String msg) {
    JOptionPane.showMessageDialog(this, msg, "Error", JOptionPane.ERROR_MESSAGE);
}

public void showInfo(String msg) {
    JOptionPane.showMessageDialog(this, msg, "Info",
JOptionPane.INFORMATION_MESSAGE);
}

public void refreshAll() {
    userVehiclePanel.refresh();
    zoneSlotPanel.refresh();
    reservationPanel.refresh();
    entryExitPanel.refresh();
    passPaymentPanel.refresh();
    violationPanel.refresh();
    reportsPanel.refresh();
    setStatus("All panels refreshed.");
}

private void testDBOnStartup() {
    SwingUtilities.invokeLater(() -> {

```

```

        String err = DBConnection.testConnection();
        if (err != null) {
            setStatus("⚠ DB connection failed - check credentials in
DBConnection.java");
            JOptionPane.showMessageDialog(this,
                "Could not connect to MySQL:\n" + err +
                "\n\nPlease:\n1. Start MySQL server\n2. Run database.sql\n3.
Update DBConnection.java with your password.",
                "Database Connection Error", JOptionPane.WARNING_MESSAGE);
        } else {
            setStatus("Connected to campus_parking database.");
        }
    });
}

private void confirmExit() {
    int r = JOptionPane.showConfirmDialog(this,
        "Exit the application?", "Confirm Exit",
        JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE);
    if (r == JOptionPane.YES_OPTION) {
        DBConnection.close();
        dispose();
        System.exit(0);
    }
}

// — MAIN —————
public static void main(String[] args) {
    try {
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
    } catch (Exception ignored) {}
    SwingUtilities.invokeLater(() -> new MainFrame().setVisible(true));
}
}

```

UserVehiclePanel.java

```

package parking.ui;

import parking.dao.UserDAO;
import parking.dao.VehicleDAO;
import parking.model.User;
import parking.model.Vehicle;

import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.*;
import java.awt.*;
import java.awt.event.*;
import java.sql.SQLException;
import java.util.List;

/**
 * UserVehiclePanel - split into two sub-tabs: Users and Vehicles.
 * Demonstrates: JTabbedPane, GridBagLayout, JTable, JScrollPane,
 * JComboBox, JTextField, JButton, input validation, event handling.
 */
public class UserVehiclePanel extends JPanel {

    private final MainFrame parent;
    private final UserDAO userDAO = new UserDAO();
    private final VehicleDAO vehicleDAO = new VehicleDAO();
}

```

```

// — User widgets —
private JTable userTable;
private DefaultTableModel userModel;
private JTextField tfUserName, tfEmail, tfPhone, tfIdNumber, tfAddress,
tfUserSearch;
private JComboBox<String> cbUserType;
private JButton btnAddUser, btnUpdateUser, btnDeleteUser, btnClearUser,
btnSearchUser;
private int selectedUserId = -1;

// — Vehicle widgets —
private JTable vehicleTable;
private DefaultTableModel vehicleModel;
private JTextField tfPlate, tfMake, tfModel, tfColor, tfVehicleSearch;
private JComboBox<String> cbVehicleType;
private JComboBox<UserItem> cbVehicleUser;
private JButton btnAddVehicle, btnUpdateVehicle, btnDeleteVehicle,
btnClearVehicle, btnSearchVehicle;
private int selectedVehicleId = -1;

public UserVehiclePanel(MainFrame parent) {
    this.parent = parent;
    setLayout(new BorderLayout());
    JTabbedPane sub = new JTabbedPane();
    sub.addTab("Users", buildUserTab());
    sub.addTab("Vehicles", buildVehicleTab());
    add(sub, BorderLayout.CENTER);
    refresh();
}

// =====
//  USER TAB
// =====
private JPanel buildUserTab() {
    JPanel panel = new JPanel(new BorderLayout(6, 6));
    panel.setBorder(new EmptyBorder(8, 8, 8, 8));

    // — Form —
    JPanel form = new JPanel(new GridBagLayout());
    form.setBorder(BorderFactory.createTitledBorder("User Details"));
    GridBagConstraints gc = new GridBagConstraints();
    gc.insets = new Insets(4, 6, 4, 6);
    gc.fill = GridBagConstraints.HORIZONTAL;

    tfUserName = new JTextField(18);
    tfEmail     = new JTextField(18);
    tfPhone     = new JTextField(14);
    tfIdNumber  = new JTextField(14);
    tfAddress   = new JTextField(24);
    cbUserType  = new JComboBox<>(new
String[] {"Student", "Faculty", "Staff", "Visitor", "Service"});

    addFormRow(form, gc, 0, "Full Name*:", tfUserName);
    addFormRow(form, gc, 1, "Email*:", tfEmail);
    addFormRow(form, gc, 2, "Phone*:", tfPhone);
    addFormRow(form, gc, 3, "ID Number*:", tfIdNumber);
    addFormRow(form, gc, 4, "User Type:", cbUserType);
    addFormRow(form, gc, 5, "Address:", tfAddress);

    // — Buttons —
    btnAddUser = makeBtn("Add", new Color(34, 139, 34));
    btnUpdateUser = makeBtn("Update", new Color(0, 102, 204));
    btnDeleteUser = makeBtn("Delete", new Color(200, 40, 40));

```

```

        btnClearUser = makeBtn("Clear", Color.GRAY);

        JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
        btns.add(btnAddUser); btns.add(btnUpdateUser);
        btns.add(btnDeleteUser); btns.add(btnClearUser);

        gc.gridx=0; gc.gridy=6; gc.gridwidth=4;
        form.add(btns, gc);

        // — Search bar —
        JPanel searchBar = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
        searchBar.add(new JLabel("Search:"));
        tfUserSearch = new JTextField(20);
        btnSearchUser = makeBtn("Go", new Color(0,102,204));
        JButton btnShowAll = makeBtn("Show All", Color.DARK_GRAY);
        searchBar.add(tfUserSearch); searchBar.add(btnSearchUser);
        searchBar.add(btnShowAll);

        JPanel top = new JPanel(new BorderLayout());
        top.add(form, BorderLayout.CENTER);
        top.add(searchBar, BorderLayout.SOUTH);
        panel.add(top, BorderLayout.NORTH);

        // — Table —
        String[] cols = {"ID", "Name", "Email", "Phone", "Type", "ID
Number", "Address", "Registered"};
        userModel = new DefaultTableModel(cols, 0) {
            public boolean isCellEditable(int r, int c){ return false; }
        };
        userTable = new JTable(userModel);
        styleTable(userTable);
        userTable.getSelectionModel().addListSelectionListener(e -> {
            if (!e.getValueIsAdjusting()) populateUserForm();
        });
        panel.add(new JScrollPane(userTable), BorderLayout.CENTER);

        // — Events —
        btnAddUser.addActionListener(e -> addUser());
        btnUpdateUser.addActionListener(e -> updateUser());
        btnDeleteUser.addActionListener(e -> deleteUser());
        btnClearUser.addActionListener(e -> clearUserForm());
        btnSearchUser.addActionListener(e -> searchUsers());
        btnShowAll.addActionListener(e -> { tfUserSearch.setText(""); loadUsers(); });
        tfUserSearch.addActionListener(e -> searchUsers());

        return panel;
    }

    private void addUser() {
        if (!validateUserForm()) return;
        try {
            User u = buildUserFromForm();
            if (userDAO.addUser(u)) {
                parent.setStatus("User added successfully.");
                clearUserForm(); loadUsers();
            }
        } catch (SQLException ex) {
            parent.showError("Add user failed:\n" + ex.getMessage());
        }
    }

    private void updateUser() {
        if (selectedUserId < 0) { parent.showError("Select a user to update.");
        return; }
    }

```



```

        if (!validateUserForm()) return;
        try {
            User u = buildUserFromForm();
            u.setUserId(selectedUserId);
            if (userDAO.updateUser(u)) {
                parent.setStatus("User updated."); clearUserForm(); loadUsers();
            }
        } catch (SQLException ex) {
            parent.showError("Update failed:\n" + ex.getMessage());
        }
    }

    private void deleteUser() {
        if (selectedUserId < 0) { parent.showError("Select a user to delete.");
        return; }
        int c = JOptionPane.showConfirmDialog(parent,
            "Delete this user and all associated vehicles/sessions?",
            "Confirm Delete", JOptionPane.YES_NO_OPTION);
        if (c != JOptionPane.YES_OPTION) return;
        try {
            userDAO.deleteUser(selectedUserId);
            parent.setStatus("User deleted."); clearUserForm(); loadUsers();
        } catch (SQLException ex) {
            parent.showError("Delete failed:\n" + ex.getMessage());
        }
    }

    private void searchUsers() {
        String kw = tfUserSearch.getText().trim();
        if (kw.isEmpty()) { loadUsers(); return; }
        try {
            List<User> list = userDAO.searchUsers(kw);
            populateUserTable(list);
            parent.setStatus(list.size() + " user(s) found.");
        } catch (SQLException ex) {
            parent.showError(ex.getMessage());
        }
    }

    private void loadUsers() {
        try { populateUserTable(userDAO.getAllUsers()); } catch (SQLException ex) {
        parent.showError(ex.getMessage()); }
    }

    private void populateUserTable(List<User> list) {
        userModel.setRowCount(0);
        for (User u : list)
            userModel.addRow(new Object[]{u.getUserId(), u.getFullName(),
            u.getEmail(),
                u.getPhone(), u.getUserType(), u.getIdNumber(), u.getAddress(),
            u.getCreatedAt()});
    }

    private void populateUserForm() {
        int row = userTable.getSelectedRow();
        if (row < 0) return;
        selectedUserId = (int) userModel.getValueAt(row, 0);
        tfUserName.setText((String) userModel.getValueAt(row, 1));
        tfEmail.setText((String) userModel.getValueAt(row, 2));
        tfPhone.setText((String) userModel.getValueAt(row, 3));
        cbUserType.setSelectedItem(userModel.getValueAt(row, 4));
        tfIdNumber.setText((String) userModel.getValueAt(row, 5));
        Object addr = userModel.getValueAt(row, 6);
        tfAddress.setText(addr != null ? addr.toString() : "");
    }

```

```

        private boolean validateUserForm() {
            if (tfUserName.getText().trim().isEmpty()) { parent.showError("Full Name is required."); return false; }
            if (tfEmail.getText().trim().isEmpty()) { parent.showError("Email is required."); return false; }
            if (tfPhone.getText().trim().isEmpty()) { parent.showError("Phone is required."); return false; }
            if (tfIdNumber.getText().trim().isEmpty()) { parent.showError("ID Number is required."); return false; }
            if (!tfEmail.getText().trim().contains("@")) { parent.showError("Enter a valid email address."); return false; }
            return true;
        }

        private User buildUserFromForm() {
            User u = new User();
            u.setFullName(tfUserName.getText().trim());
            u.setEmail(tfEmail.getText().trim());
            u.setPhone(tfPhone.getText().trim());
            u.setUserType((String) cbUserType.getSelectedItem());
            u.setIdNumber(tfIdNumber.getText().trim());
            u.setAddress(tfAddress.getText().trim());
            return u;
        }

        private void clearUserForm() {
            selectedUserId = -1;
            tfUserName.setText(""); tfEmail.setText(""); tfPhone.setText("");
            tfIdNumber.setText(""); tfAddress.setText("");
            cbUserType.setSelectedIndex(0);
            userTable.clearSelection();
        }

// =====
//  VEHICLE TAB
// =====
        private JPanel buildVehicleTab() {
            JPanel panel = new JPanel(new BorderLayout(6, 6));
            panel.setBorder(new EmptyBorder(8, 8, 8, 8));

            JPanel form = new JPanel(new GridBagLayout());
            form.setBorder(BorderFactory.createTitledBorder("Vehicle Details"));
            GridBagConstraints gc = new GridBagConstraints();
            gc.insets = new Insets(4, 6, 4, 6);
            gc.fill = GridBagConstraints.HORIZONTAL;

            cbVehicleUser = new JComboBox<>();
            tfPlate = new JTextField(14);
            cbVehicleType = new JComboBox<>(new
String[]{"Car", "Motorcycle", "Truck", "Bus", "Bicycle", "Other"});
            tfMake = new JTextField(14);
            tfModel = new JTextField(14);
            tfColor = new JTextField(10);

            addFormRow(form, gc, 0, "Owner (User)*:", cbVehicleUser);
            addFormRow(form, gc, 1, "License Plate*:", tfPlate);
            addFormRow(form, gc, 2, "Vehicle Type:", cbVehicleType);
            addFormRow(form, gc, 3, "Make:", tfMake);
            addFormRow(form, gc, 4, "Model:", tfModel);
            addFormRow(form, gc, 5, "Color:", tfColor);

            btnAddVehicle = makeBtn("Add", new Color(34, 139, 34));
            btnUpdateVehicle = makeBtn("Update", new Color(0, 102, 204));

```

```

        btnDeleteVehicle = makeBtn("Delete", new Color(200, 40, 40));
        btnClearVehicle = makeBtn("Clear", Color.GRAY);

        JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
        btns.add(btnAddVehicle); btns.add(btnUpdateVehicle);
        btns.add(btnDeleteVehicle); btns.add(btnClearVehicle);
        gc.gridx=0; gc.gridy=6; gc.gridwidth=4;
        form.add(btns, gc);

        JPanel searchBar = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
        searchBar.add(new JLabel("Search:"));
        tfVehicleSearch = new JTextField(20);
        btnSearchVehicle = makeBtn("Go", new Color(0,102,204));
        JButton btnShowAll = makeBtn("Show All", Color.DARK_GRAY);
        searchBar.add(tfVehicleSearch); searchBar.add(btnSearchVehicle);
        searchBar.add(btnShowAll);

        JPanel top = new JPanel(new BorderLayout());
        top.add(form, BorderLayout.CENTER);
        top.add(searchBar, BorderLayout.SOUTH);
        panel.add(top, BorderLayout.NORTH);

        String[] cols = {"ID", "Owner", "License
Plate", "Type", "Make", "Model", "Color", "Registered"};
        vehicleModel = new DefaultTableModel(cols, 0) {
            public boolean isCellEditable(int r, int c){ return false; }
        };
        vehicleTable = new JTable(vehicleModel);
        styleTable(vehicleTable);
        vehicleTable.getSelectionModel().addListSelectionListener(e -> {
            if (!e.getValueIsAdjusting()) populateVehicleForm();
        });
        panel.add(new JScrollPane(vehicleTable), BorderLayout.CENTER);

        btnAddVehicle.addActionListener(e -> addVehicle());
        btnUpdateVehicle.addActionListener(e -> updateVehicle());
        btnDeleteVehicle.addActionListener(e -> deleteVehicle());
        btnClearVehicle.addActionListener(e -> clearVehicleForm());
        btnSearchVehicle.addActionListener(e -> searchVehicles());
        btnShowAll.addActionListener(e -> { tfVehicleSearch.setText("");
        loadVehicles(); });
        tfVehicleSearch.addActionListener(e -> searchVehicles());

        return panel;
    }

    private void addVehicle() {
        if (!validateVehicleForm()) return;
        try {
            Vehicle v = buildVehicleFromForm();
            if (vehicleDAO.addVehicle(v)) {
                parent.setStatus("Vehicle registered."); clearVehicleForm();
                loadVehicles();
            }
        } catch (SQLException ex) {
            parent.showError("Add vehicle failed:\n" + ex.getMessage());
        }
    }

    private void updateVehicle() {
        if (selectedVehicleId < 0) { parent.showError("Select a vehicle to update.");
        return; }
        if (!validateVehicleForm()) return;
        try {

```

```

        Vehicle v = buildVehicleFromForm();
        v.setVehicleId(selectedVehicleId);
        if (vehicleDAO.updateVehicle(v)) {
            parent.setStatus("Vehicle updated."); clearVehicleForm();
loadVehicles();
        }
    } catch (SQLException ex) { parent.showError("Update failed:\n" +
ex.getMessage()); }
    }

    private void deleteVehicle() {
        if (selectedVehicleId < 0) { parent.showError("Select a vehicle."); return; }
        int c = JOptionPane.showConfirmDialog(parent, "Delete this vehicle?",
"Confirm", JOptionPane.YES_NO_OPTION);
        if (c != JOptionPane.YES_OPTION) return;
        try {
            vehicleDAO.deleteVehicle(selectedVehicleId);
            parent.setStatus("Vehicle deleted."); clearVehicleForm(); loadVehicles();
        } catch (SQLException ex) { parent.showError("Delete failed:\n" +
ex.getMessage()); }
    }

    private void searchVehicles() {
        String kw = tfVehicleSearch.getText().trim();
        if (kw.isEmpty()) { loadVehicles(); return; }
        try {
            List<Vehicle> list = vehicleDAO.searchVehicles(kw);
            populateVehicleTable(list);
            parent.setStatus(list.size() + " vehicle(s) found.");
        } catch (SQLException ex) { parent.showError(ex.getMessage()); }
    }

    private void loadVehicles() {
        try { populateVehicleTable(vehicleDAO.getAllVehicles()); } catch (SQLException
ex) { parent.showError(ex.getMessage()); }
    }

    private void populateVehicleTable(List<Vehicle> list) {
        vehicleModel.setRowCount(0);
        for (Vehicle v : list)
            vehicleModel.addRow(new Object[]{v.getVehicleId(), v.getOwnerName(),
v.getLicensePlate(),
            v.getVehicleType(), v.getMake(), v.getModel(), v.getColor(),
v.getRegisteredAt()});
    }

    private void populateVehicleForm() {
        int row = vehicleTable.getSelectedRow();
        if (row < 0) return;
        selectedVehicleId = (int) vehicleModel.getValueAt(row, 0);
        tfPlate.setText((String) vehicleModel.getValueAt(row, 2));
        cbVehicleType.setSelectedItem(vehicleModel.getValueAt(row, 3));
        tfMake.setText(str(vehicleModel.getValueAt(row, 4)));
        tfModel.setText(str(vehicleModel.getValueAt(row, 5)));
        tfColor.setText(str(vehicleModel.getValueAt(row, 6)));
    }

    private boolean validateVehicleForm() {
        if (cbVehicleUser.getSelectedIndex() < 0) { parent.showError("Select an
owner."); return false; }
        if (tfPlate.getText().trim().isEmpty()) { parent.showError("License plate
is required."); return false; }
        return true;
    }

```

```

private Vehicle buildVehicleFromForm() {
    Vehicle v = new Vehicle();
    UserItem ui = (UserItem) cbVehicleUser.getSelectedItem();
    if (ui != null) v.setUserId(ui.id);
    v.setLicensePlate(tfPlate.getText().trim().toUpperCase());
    v.setVehicleType((String) cbVehicleType.getSelectedItem());
    v.setMake(tfMake.getText().trim());
    v.setModel(tfModel.getText().trim());
    v.setColor(tfColor.getText().trim());
    return v;
}

private void clearVehicleForm() {
    selectedVehicleId = -1;
    tfPlate.setText(""); tfMake.setText(""); tfModel.setText("");
    tfColor.setText("");
    cbVehicleType.setSelectedIndex(0);
    if (cbVehicleUser.getItemCount() > 0) cbVehicleUser.setSelectedIndex(0);
    vehicleTable.clearSelection();
}

private void loadUserCombo() {
    cbVehicleUser.removeAllItems();
    try {
        for (User u : userDAO.getAllUsers())
            cbVehicleUser.addItem(new UserItem(u.getUserId(), u.getFullName() + "
(" + u.getUserType() + ")"));
    } catch (SQLException ex) { parent.showError(ex.getMessage()); }
}

// — Shared helpers —————
public void refresh() { loadUsers(); loadVehicles(); loadUserCombo(); }

private void addFormRow(JPanel p, GridBagConstraints gc, int row, String label,
JComponent field) {
    gc.gridx=0; gc.gridy=row; gc.gridwidth=1; gc.weightx=0;
    p.add(new JLabel(label), gc);
    gc.gridx=1; gc.weightx=1; gc.gridwidth=3;
    p.add(field, gc);
}

private void styleTable(JTable t) {
    t.setRowHeight(22);
    t.setFont(new Font("Segoe UI", Font.PLAIN, 12));
    t.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 12));
    t.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
    t.setAutoCreateRowSorter(true);
}

private JButton makeBtn(String text, Color bg) {
    JButton b = new JButton(text);
    b.setBackground(bg);
    b.setForeground(Color.WHITE);
    b.setOpaque(true);
    b.setBorderPainted(false);
    b.setFocusPainted(false);
    b.setFont(new Font("Segoe UI", Font.BOLD, 12));
    return b;
}

private String str(Object o) { return o != null ? o.toString() : ""; }

/** ComboBox item wrapper for users */

```

```

static class UserItem {
    final int id; final String label;
    UserItem(int id, String label){ this.id=id; this.label=label; }
    public String toString(){ return label; }
}
}

```

ZoneSlotPanel.java

```

package parking.ui;

import parking.dao.ParkingZoneDAO;
import parking.dao.ParkingSlotDAO;
import parking.model.ParkingZone;
import parking.model.ParkingSlot;

import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.*;
import java.awt.*;
import java.math.BigDecimal;
import java.sql.SQLException;
import java.util.List;

/**
 * ZoneSlotPanel - manage parking zones and their slots.
 * Demonstrates: JSplitPane, JTable with colour-coded rows, GridBagLayout,
 * JSpinner, JComboBox, PreparedStatement-backed DAO calls.
 */
public class ZoneSlotPanel extends JPanel {

    private final MainFrame parent;
    private final ParkingZoneDAO zoneDAO = new ParkingZoneDAO();
    private final ParkingSlotDAO slotDAO = new ParkingSlotDAO();

    // Zone widgets
    private JTable zoneTable;
    private DefaultTableModel zoneModel;
    private JTextField tfZoneName, tfLocation, tfDescription;
    private JComboBox<String> cbZoneType;
    private JSpinner spnTotalSlots;
    private JTextField tfHourlyRate;
    private JButton btnAddZone, btnUpdateZone, btnDeleteZone, btnClearZone;
    private int selectedZoneId = -1;

    // Slot widgets
    private JTable slotTable;
    private DefaultTableModel slotModel;
    private JComboBox<ZoneItem> cbSlotZone;
    private JTextField tfSlotNumber;
    private JComboBox<String> cbSlotType, cbSlotStatus, cbFilterStatus;
    private JButton btnAddSlot, btnUpdateSlot, btnDeleteSlot, btnClearSlot;
    private int selectedSlotId = -1;

    public ZoneSlotPanel(MainFrame parent) {
        this.parent = parent;
        setLayout(new BorderLayout());
        JSplitPane split = new JSplitPane(JSplitPane.VERTICAL_SPLIT, buildZonePanel(),
        buildSlotPanel());
        split.setResizeWeight(0.45);
        split.setDividerSize(6);
        add(split, BorderLayout.CENTER);
    }

```

```

        refresh();
    }

    // =====
    // ZONE PANEL
    // =====
    private JPanel buildZonePanel() {
        JPanel panel = new JPanel(new BorderLayout(6, 6));
        panel.setBorder(new TitledBorder("Parking Zones"));

        // Form
        JPanel form = new JPanel(new GridBagLayout());
        GridBagConstraints gc = gbc();

        tfZoneName = new JTextField(14);
        cbZoneType = new JComboBox<>(new
String[] {"Student", "Faculty", "Staff", "Visitor", "Service", "General"});
        tfLocation = new JTextField(18);
        spnTotalSlots = new JSpinner(new SpinnerNumberModel(10, 1, 500, 1));
        tfHourlyRate = new JTextField("0.00", 8);
        tfDescription = new JTextField(22);

        row(form, gc, 0, "Zone Name*:", tfZoneName);
        row(form, gc, 1, "Zone Type:", cbZoneType);
        row(form, gc, 2, "Location:", tfLocation);
        row(form, gc, 3, "Total Slots:", spnTotalSlots);
        row(form, gc, 4, "Hourly Rate (₹):", tfHourlyRate);
        row(form, gc, 5, "Description:", tfDescription);

        btnAddZone = btn("Add Zone", new Color(34,139,34));
        btnUpdateZone = btn("Update", new Color(0,102,204));
        btnDeleteZone = btn("Delete", new Color(200,40,40));
        btnClearZone = btn("Clear", Color.GRAY);

        JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
        btns.add(btnAddZone); btns.add(btnUpdateZone); btns.add(btnDeleteZone);
        btns.add(btnClearZone);
        gc.gridx=0; gc.gridy=6; gc.gridwidth=4; form.add(btns, gc);

        panel.add(form, BorderLayout.WEST);

        // Table
        String[] cols = {"ID", "Zone Name", "Type", "Location", "Total", "Available", "Rate
(₹)", "Description"};
        zoneModel = new DefaultTableModel(cols, 0){ public boolean isCellEditable(int
r,int c){return false;} };
        zoneTable = new JTable(zoneModel);
        styleTable(zoneTable);
        zoneTable.getSelectionModel().addListSelectionListener(e -> {
if(!e.getValueIsAdjusting()) fillZoneForm(); });
        panel.add(new JScrollPane(zoneTable), BorderLayout.CENTER);

        btnAddZone.addActionListener(e -> addZone());
        btnUpdateZone.addActionListener(e -> updateZone());
        btnDeleteZone.addActionListener(e -> deleteZone());
        btnClearZone.addActionListener(e -> clearZoneForm());

        return panel;
    }

    private void addZone() {
        if (!validateZoneForm()) return;
        try {

```

```

        ParkingZone z = buildZoneFromForm();
        if (zoneDAO.addZone(z)) { parent.setStatus("Zone added.");
clearZoneForm(); loadZones(); loadZoneCombo(); }
        } catch (SQLException ex) { parent.showError("Add zone failed:\n" +
ex.getMessage()); }
    }

    private void updateZone() {
        if (selectedZoneId < 0) { parent.showError("Select a zone."); return; }
        if (!validateZoneForm()) return;
        try {
            ParkingZone z = buildZoneFromForm(); z.setZoneId(selectedZoneId);
            if (zoneDAO.updateZone(z)) { parent.setStatus("Zone updated.");
clearZoneForm(); loadZones(); loadZoneCombo(); }
        } catch (SQLException ex) { parent.showError("Update failed:\n" +
ex.getMessage()); }
    }

    private void deleteZone() {
        if (selectedZoneId < 0) { parent.showError("Select a zone."); return; }
        int c = JOptionPane.showConfirmDialog(parent, "Delete this zone and all its
slots?", "Confirm", JOptionPane.YES_NO_OPTION);
        if (c!=JOptionPane.YES_OPTION) return;
        try { zoneDAO.deleteZone(selectedZoneId); parent.setStatus("Zone deleted.");
clearZoneForm(); loadZones(); loadZoneCombo(); }
        catch (SQLException ex) { parent.showError("Delete failed:\n" +
ex.getMessage()); }
    }

    private void fillZoneForm() {
        int row = zoneTable.getSelectedRow(); if(row<0) return;
        selectedZoneId = (int) zoneModel.getValueAt(row,0);
        tfZoneName.setText((String) zoneModel.getValueAt(row,1));
        cbZoneType.setSelectedItem(zoneModel.getValueAt(row,2));
        tfLocation.setText(str(zoneModel.getValueAt(row,3)));
        spnTotalSlots.setValue(zoneModel.getValueAt(row,4));
        tfHourlyRate.setText(str(zoneModel.getValueAt(row,6)));
        tfDescription.setText(str(zoneModel.getValueAt(row,7)));
        // load slots for this zone
        loadSlotsForZone(selectedZoneId);
    }

    private void clearZoneForm() {
        selectedZoneId=-1; tfZoneName.setText(""); tfLocation.setText("");
        tfHourlyRate.setText("0.00"); tfDescription.setText("");
        spnTotalSlots.setValue(10); cbZoneType.setSelectedIndex(0);
        zoneTable.clearSelection();
    }

    private boolean validateZoneForm() {
        if (tfZoneName.getText().trim().isEmpty()) { parent.showError("Zone name
required."); return false; }
        try { new BigDecimal(tfHourlyRate.getText().trim()); } catch
(NumberFormatException e) { parent.showError("Invalid hourly rate."); return false; }
        return true;
    }

    private ParkingZone buildZoneFromForm() {
        ParkingZone z = new ParkingZone();
        z.setZoneName(tfZoneName.getText().trim());
        z.setZoneType((String) cbZoneType.getSelectedItem());
        z.setLocation(tfLocation.getText().trim());
        z.setTotalSlots((int) spnTotalSlots.getValue());
        z.setHourlyRate(new BigDecimal(tfHourlyRate.getText().trim()));
        z.setDescription(tfDescription.getText().trim());
    }

```



```

        return z;
    }

    private void loadZones() {
        try {
            zoneModel.setRowCount(0);
            for (ParkingZone z : zoneDAO.getAllZones())
                zoneModel.addRow(new
Object[] {z.getZoneId(), z.getZoneName(), z.getZoneType(), z.getLocation(),
z.getTotalSlots(), z.getAvailableSlots(), z.getHourlyRate(), z.getDescription()});
        } catch (SQLException ex) { parent.showError(ex.getMessage()); }
    }

    // =====
    // SLOT PANEL
    // =====
    private JPanel buildSlotPanel() {
        JPanel panel = new JPanel(new BorderLayout(6, 6));
        panel.setBorder(new TitledBorder("Parking Slots"));

        JPanel form = new JPanel(new GridBagLayout());
        GridBagConstraints gc = gbc();

        cbSlotZone = new JComboBox<>();
        tfSlotNumber = new JTextField(8);
        cbSlotType = new JComboBox<>(new
String[] {"Regular", "Handicapped", "EV", "Reserved"});
        cbSlotStatus = new JComboBox<>(new
String[] {"Available", "Occupied", "Reserved", "Maintenance"});
        cbFilterStatus = new JComboBox<>(new
String[] {"All", "Available", "Occupied", "Reserved", "Maintenance"});

        row(form, gc, 0, "Zone*:", cbSlotZone);
        row(form, gc, 1, "Slot Number*:", tfSlotNumber);
        row(form, gc, 2, "Slot Type:", cbSlotType);
        row(form, gc, 3, "Status:", cbSlotStatus);
        row(form, gc, 4, "Filter Status:", cbFilterStatus);

        btnAddSlot = btn("Add Slot", new Color(34,139,34));
        btnUpdateSlot = btn("Update", new Color(0,102,204));
        btnDeleteSlot = btn("Delete", new Color(200,40,40));
        btnClearSlot = btn("Clear", Color.GRAY);
        JButton btnFilter = btn("Filter", new Color(120,60,180));

        JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
        btns.add(btnAddSlot); btns.add(btnUpdateSlot); btns.add(btnDeleteSlot);
        btns.add(btnClearSlot); btns.add(btnFilter);
        gc.gridx=0; gc.gridy=5; gc.gridwidth=4; form.add(btns, gc);

        panel.add(form, BorderLayout.WEST);

        String[] cols = {"ID", "Zone", "Slot No", "Type", "Status"};
        slotModel = new DefaultTableModel(cols, 0) { public boolean isCellEditable(int
r, int c) { return false; } };
        slotTable = new JTable(slotModel) {
            public Component prepareRenderer(TableCellRenderer r, int row, int col) {
                Component c = super.prepareRenderer(r, row, col);
                String status = str(slotModel.getValueAt(row, 4));
                if (!isRowSelected(row)) {
                    switch(status) {
                        case "Available": c.setBackground(new Color(220,255,220));
break;

```

```

        case "Occupied":      c.setBackground(new Color(255,220,220));
break;
        case "Reserved":     c.setBackground(new Color(255,255,200));
break;
        case "Maintenance":  c.setBackground(new Color(220,220,220));
break;
        default:              c.setBackground(Color.WHITE);
    }
    return c;
}
};
styleTable(slotTable);
slotTable.getSelectionModel().addListSelectionListener(e -> {
if(!e.getValueIsAdjusting()) fillSlotForm(); });
panel.add(new JScrollPane(slotTable), BorderLayout.CENTER);

// Legend
JPanel legend = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 2));
legend.add(colorLabel("Available", new Color(220,255,220)));
legend.add(colorLabel("Occupied", new Color(255,220,220)));
legend.add(colorLabel("Reserved", new Color(255,255,200)));
legend.add(colorLabel("Maintenance", new Color(220,220,220)));
panel.add(legend, BorderLayout.SOUTH);

btnAddSlot.addActionListener(e -> addSlot());
btnUpdateSlot.addActionListener(e -> updateSlot());
btnDeleteSlot.addActionListener(e -> deleteSlot());
btnClearSlot.addActionListener(e -> clearSlotForm());
btnFilter.addActionListener(e -> filterSlots());
cbSlotZone.addActionListener(e -> {
    ZoneItem zi = (ZoneItem) cbSlotZone.getSelectedItem();
    if(zi!=null) loadSlotsForZone(zi.id);
});

return panel;
}

private void addSlot() {
    if(!validateSlotForm()) return;
    try {
        ParkingSlot s = buildSlotFromForm();
        if(slotDAO.addSlot(s)) { parent.setStatus("Slot added."); clearSlotForm();
        ZoneItem zi=(ZoneItem)cbSlotZone.getSelectedItem(); if(zi!=null)
        loadSlotsForZone(zi.id); }
    } catch(SQLException ex){ parent.showError("Add slot
failed:\n"+ex.getMessage()); }
}

private void updateSlot() {
    if(selectedSlotId<0){ parent.showError("Select a slot."); return; }
    if(!validateSlotForm()) return;
    try {
        ParkingSlot s = buildSlotFromForm(); s.setSlotId(selectedSlotId);
        if(slotDAO.updateSlot(s)){ parent.setStatus("Slot updated.");
        clearSlotForm(); ZoneItem zi=(ZoneItem)cbSlotZone.getSelectedItem(); if(zi!=null)
        loadSlotsForZone(zi.id); }
    } catch(SQLException ex){ parent.showError("Update
failed:\n"+ex.getMessage()); }
}

private void deleteSlot() {
    if(selectedSlotId<0){ parent.showError("Select a slot."); return; }
    int c=JOptionPane.showConfirmDialog(parent,"Delete this
slot?", "Confirm",JOptionPane.YES_NO_OPTION);

```

```

        if(c!=JOptionPane.YES_OPTION) return;
        try{ slotDAO.deleteSlot(selectedSlotId); parent.setStatus("Slot deleted.");
        clearSlotForm(); ZoneItem zi=(ZoneItem)cbSlotZone.getSelectedItemAt(); if(zi!=null)
        loadSlotsForZone(zi.id); }
        catch(SQLException ex){ parent.showError(ex.getMessage()); }
    }

    private void filterSlots() {
        String status = (String) cbFilterStatus.getSelectedItemAt();
        try {
            slotModel.setRowCount(0);
            List<ParkingSlot> list = "All".equals(status) ? slotDAO.getAllSlots() :
slotDAO.getSlotsByStatus(status);
            for(ParkingSlot s : list)
                slotModel.addRow(new
Object[]{s.getSlotId(),s.getZoneName(),s.getSlotNumber(),s.getSlotType(),s.getStatus()
});
            parent.setStatus(list.size()+" slot(s) shown.");
        } catch(SQLException ex){ parent.showError(ex.getMessage()); }
    }

    private void loadSlotsForZone(int zoneId) {
        try {
            slotModel.setRowCount(0);
            for(ParkingSlot s : slotDAO.getSlotsByZone(zoneId))
                slotModel.addRow(new
Object[]{s.getSlotId(),s.getZoneName(),s.getSlotNumber(),s.getSlotType(),s.getStatus()
});
        } catch(SQLException ex){ parent.showError(ex.getMessage()); }
    }

    private void fillSlotForm() {
        int row = slotTable.getSelectedRow(); if(row<0) return;
        selectedSlotId = (int) slotModel.getValueAt(row,0);
        tfSlotNumber.setText((String)slotModel.getValueAt(row,2));
        cbSlotType.setSelectedItemAt(slotModel.getValueAt(row,3));
        cbSlotStatus.setSelectedItemAt(slotModel.getValueAt(row,4));
    }

    private void clearSlotForm() {
        selectedSlotId=-1; tfSlotNumber.setText("");
        cbSlotType.setSelectedIndex(0); cbSlotStatus.setSelectedIndex(0);
        slotTable.clearSelection();
    }

    private boolean validateSlotForm() {
        if(cbSlotZone.getSelectedIndex()<0){ parent.showError("Select a zone.");
return false; }
        if(tfSlotNumber.getText().trim().isEmpty()){ parent.showError("Slot number
required."); return false; }
        return true;
    }

    private ParkingSlot buildSlotFromForm() {
        ParkingSlot s = new ParkingSlot();
        ZoneItem zi = (ZoneItem) cbSlotZone.getSelectedItemAt();
        if(zi!=null) s.setZoneId(zi.id);
        s.setSlotNumber(tfSlotNumber.getText().trim().toUpperCase());
        s.setSlotType((String)cbSlotType.getSelectedItemAt());
        s.setStatus((String)cbSlotStatus.getSelectedItemAt());
        return s;
    }

    private void loadZoneCombo() {

```

```

        cbSlotZone.removeAllItems();
        try { for(ParkingZone z : zoneDAO.getAllZones()) cbSlotZone.addItem(new
ZoneItem(z.getId(), z.getName()+" (" + z.getType() + ")")); }
        catch(SQLException ex){ parent.showError(ex.getMessage()); }
    }

    // — Shared —————
    public void refresh() { loadZones(); loadZoneCombo(); }

    private GridBagConstraints gbc() {
        GridBagConstraints gc = new GridBagConstraints();
        gc.insets = new Insets(4,6,4,6); gc.fill=GridBagConstraints.HORIZONTAL; return
gc;
    }
    private void row(JPanel p, GridBagConstraints gc, int r, String lbl, JComponent f)
    {
        gc.gridx=0; gc.gridy=r; gc.gridwidth=1; gc.weightx=0; p.add(new
JLabel(lbl),gc);
        gc.gridx=1; gc.weightx=1; gc.gridwidth=3; p.add(f,gc);
    }
    private void styleTable(JTable t) {
        t.setRowHeight(22); t.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        t.setFont(new Font("Segoe UI",Font.PLAIN,12));
        t.getTableHeader().setFont(new Font("Segoe UI",Font.BOLD,12));
        t.setAutoCreateRowSorter(true);
    }
    private JButton btn(String text, Color bg) {
        JButton b=new JButton(text); b.setBackground(bg);
b.setForeground(Color.WHITE);
        b.setOpaque(true); b.setBorderPainted(false);
        b.setFocusPainted(false); b.setFont(new Font("Segoe UI",Font.BOLD,12)); return
b;
    }
    private JLabel colorLabel(String text, Color bg) {
        JLabel l = new JLabel(" "+text+" "); l.setOpaque(true); l.setBackground(bg);
        l.setBorder(BorderFactory.createLineBorder(Color.GRAY)); return l;
    }
    private String str(Object o){ return o!=null?o.toString():""; }

    static class ZoneItem { final int id; final String label;
        ZoneItem(int id,String label){this.id=id;this.label=label;}
        public String toString(){return label;} }
}

```

ReservationPanel.java

```

package parking.ui;

import parking.dao.*;
import parking.model.*;

import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.*;
import java.awt.*;
import java.sql.*;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;

/**
 * ReservationPanel - create/cancel/view reservations.

```

```

* Demonstrates: JComboBox chaining, JTextField date input,
* conflict-checking PreparedStatement, status filtering.
*/
public class ReservationPanel extends JPanel {

    private final MainFrame parent;
    private final ReservationDAO reservationDAO = new ReservationDAO();
    private final UserDAO userDAO = new UserDAO();
    private final VehicleDAO vehicleDAO = new VehicleDAO();
    private final ParkingZoneDAO zoneDAO = new ParkingZoneDAO();
    private final ParkingSlotDAO slotDAO = new ParkingSlotDAO();

    private JComboBox<UserItem>    cbUser;
    private JComboBox<VehicleItem> cbVehicle;
    private JComboBox<ZoneItem>    cbZone;
    private JComboBox<SlotItem>    cbSlot;
    private JTextField tfFrom, tfUntil;
    private JLabel lblAvailable;
    private JButton btnReserve, btnCancel, btnRefresh;
    private JComboBox<String> cbStatusFilter;

    private JTable table;
    private DefaultTableModel tableModel;
    private int selectedReservationId = -1;

    private static final String DATE_FMT = "yyyy-MM-dd HH:mm";

    public ReservationPanel(MainFrame parent) {
        this.parent = parent;
        setLayout(new BorderLayout(6, 6));
        setBorder(new EmptyBorder(8, 8, 8, 8));
        add(buildForm(), BorderLayout.NORTH);
        add(buildTable(), BorderLayout.CENTER);
        refresh();
    }

    private JPanel buildForm() {
        JPanel form = new JPanel(new GridBagLayout());
        form.setBorder(new TitledBorder("New Reservation"));
        GridBagConstraints gc = new GridBagConstraints();
        gc.insets = new Insets(5, 8, 5, 8);
        gc.fill = GridBagConstraints.HORIZONTAL;

        cbUser    = new JComboBox<>();
        cbVehicle = new JComboBox<>();
        cbZone    = new JComboBox<>();
        cbSlot    = new JComboBox<>();
        lblAvailable = new JLabel("Available slots: -");

        SimpleDateFormat sdf = new SimpleDateFormat(DATE_FMT);
        tfFrom = new JTextField(sdf.format(new Date()), 16);
        tfUntil = new JTextField(sdf.format(new Date(System.currentTimeMillis() +
3600_000L)), 16);

        // Row 0
        gc.gridx=0; gc.gridy=0; gc.weightx=0; form.add(new JLabel("User*:"), gc);
        gc.gridx=1; gc.weightx=1; form.add(cbUser, gc);
        gc.gridx=2; gc.weightx=0; form.add(new JLabel("Vehicle*:"), gc);
        gc.gridx=3; gc.weightx=1; form.add(cbVehicle, gc);

        // Row 1
        gc.gridx=0; gc.gridy=1; gc.weightx=0; form.add(new JLabel("Zone*:"), gc);
        gc.gridx=1; gc.weightx=1; form.add(cbZone, gc);

```

```

        gc.gridx=2; gc.weightx=0; form.add(new JLabel("Slot*:"), gc);
        gc.gridx=3; gc.weightx=1; form.add(cbSlot, gc);

        // Row 2
        gc.gridx=0; gc.gridy=2; gc.weightx=0; form.add(new JLabel("From (yyyy-MM-dd
HH:mm)*:"), gc);
        gc.gridx=1; gc.weightx=1; form.add(tfFrom, gc);
        gc.gridx=2; gc.weightx=0; form.add(new JLabel("Until (yyyy-MM-dd HH:mm)*:"),
gc);
        gc.gridx=3; gc.weightx=1; form.add(tfUntil, gc);

        // Row 3 - availability info + buttons
        gc.gridx=0; gc.gridy=3; gc.gridwidth=2; form.add(lblAvailable, gc);
        JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 0));
        btnReserve = makeBtn("Reserve Slot", new Color(34,139,34));
        btnCancel = makeBtn("Cancel Selected", new Color(200,40,40));
        btnRefresh = makeBtn("Refresh", Color.DARK_GRAY);
        cbStatusFilter = new JComboBox<>(new
String[]{"All","Active","Completed","Cancelled","Expired"});
        btns.add(btnReserve); btns.add(btnCancel); btns.add(btnRefresh);
        btns.add(new JLabel(" Filter:")); btns.add(cbStatusFilter);
        gc.gridx=2; gc.gridwidth=2; form.add(btns, gc);

        // Events
        cbUser.addActionListener(e -> reloadVehicles());
        cbZone.addActionListener(e -> reloadSlots());
        btnReserve.addActionListener(e -> makeReservation());
        btnCancel.addActionListener(e -> cancelReservation());
        btnRefresh.addActionListener(e -> refresh());
        cbStatusFilter.addActionListener(e -> loadReservations());

        return form;
    }

    private JScrollPane buildTable() {
        String[] cols = {"ID","User","License
Plate","Zone","Slot","From","Until","Status","Created"};
        tableModel = new DefaultTableModel(cols, 0){ public boolean isCellEditable(int
r,int c){return false;} };
        table = new JTable(tableModel);
        table.setRowHeight(22);
        table.setFont(new Font("Segoe UI", Font.PLAIN, 12));
        table.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 12));
        table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        table.setAutoCreateRowSorter(true);
        table.getSelectionModel().addListSelectionListener(e -> {
            if (!e.getValueIsAdjusting()) {
                int row = table.getSelectedRow();
                selectedReservationId = row >= 0 ? (int) tableModel.getValueAt(row, 0)
: -1;
            }
        });
        return new JScrollPane(table);
    }

    private void makeReservation() {
        if (cbUser.getSelectedIndex()<0)    { parent.showError("Select a user.");
return; }
        if (cbVehicle.getSelectedIndex()<0) { parent.showError("Select a vehicle.");
return; }
        if (cbZone.getSelectedIndex()<0)    { parent.showError("Select a zone.");
return; }
        if (cbSlot.getSelectedIndex()<0)    { parent.showError("Select a slot.");
return; }
    }

```

```

        Timestamp from, until;
        try {
            SimpleDateFormat sdf = new SimpleDateFormat(DATE_FMT);
            from = new Timestamp(sdf.parse(tfFrom.getText().trim()).getTime());
            until = new Timestamp(sdf.parse(tfUntil.getText().trim()).getTime());
        } catch (Exception ex) {
            parent.showError("Invalid date format. Use: " + DATE_FMT); return;
        }
        if (!until.after(from)) { parent.showError("'Until' must be after 'From.'");
return; }

        Reservation r = new Reservation();
        r.setUserId(((UserItem)cbUser.getSelectedItem()).id);
        r.setVehicleId(((VehicleItem)cbVehicle.getSelectedItem()).id);
        r.setSlotId(((SlotItem)cbSlot.getSelectedItem()).id);
        r.setReservedFrom(from);
        r.setReservedUntil(until);

        try {
            int id = reservationDAO.addReservation(r);
            parent.setStatus("Reservation created. ID = " + id);
            refresh();
        } catch (SQLException ex) {
            parent.showError("Reservation failed:\n" + ex.getMessage());
        }
    }

    private void cancelReservation() {
        if (selectedReservationId < 0) { parent.showError("Select a reservation to
cancel."); return; }
        int c = JOptionPane.showConfirmDialog(parent, "Cancel this
reservation?", "Confirm", JOptionPane.YES_NO_OPTION);
        if (c!=JOptionPane.YES_OPTION) return;
        try {
            reservationDAO.cancelReservation(selectedReservationId);
            parent.setStatus("Reservation cancelled."); refresh();
        } catch (SQLException ex) { parent.showError(ex.getMessage()); }
    }

    private void reloadVehicles() {
        cbVehicle.removeAllItems();
        UserItem ui = (UserItem) cbUser.getSelectedItem();
        if (ui == null) return;
        try {
            for (Vehicle v : vehicleDAO.getVehiclesByUser(ui.id))
                cbVehicle.addItem(new VehicleItem(v.getVehicleId(),
v.getLicensePlate()+" ("+v.getVehicleType()+")"));
        } catch (SQLException ex) { parent.showError(ex.getMessage()); }
    }

    private void reloadSlots() {
        cbSlot.removeAllItems();
        ZoneItem zi = (ZoneItem) cbZone.getSelectedItem();
        if (zi == null) return;
        try {
            List<ParkingSlot> available = slotDAO.getAvailableSlots(zi.id);
            lblAvailable.setText("Available slots: " + available.size());
            for (ParkingSlot s : available)
                cbSlot.addItem(new SlotItem(s.getSlotId(), s.getSlotNumber()+"
["+s.getSlotType()+"]"));
        } catch (SQLException ex) { parent.showError(ex.getMessage()); }
    }

```

```

private void loadReservations() {
    try {
        tableModel.setRowCount(0);
        List<Reservation> list = reservationDAO.getAllReservations();
        String filter = (String) cbStatusFilter.getSelectedItem();
        for (Reservation r : list) {
            if (!"All".equals(filter) && !filter.equals(r.getStatus())) continue;
            tableModel.addRow(new Object[]{r.getReservationId(), r.getUserName(),
r.getLicensePlate(),
                r.getZoneName(), r.getSlotNumber(), r.getReservedFrom(),
r.getReservedUntil(),
                r.getStatus(), r.getCreatedAt()});
        }
    } catch (SQLException ex) { parent.showError(ex.getMessage()); }
}

private void loadCombos() {
    cbUser.removeAllItems();
    cbZone.removeAllItems();
    try {
        for (User u : userDAO.getAllUsers())
            cbUser.addItem(new UserItem(u.getUserId(), u.getFullName()+"
"+u.getUserType()+""));
        for (ParkingZone z : zoneDAO.getAllZones())
            cbZone.addItem(new ZoneItem(z.getZoneId(), z.getZoneName()+" -
"+z.getZoneType()));
    } catch (SQLException ex) { parent.showError(ex.getMessage()); }
    reloadVehicles();
    reloadSlots();
}

public void refresh() { loadCombos(); loadReservations(); }

private JButton makeBtn(String t, Color bg) {
    JButton b=new JButton(t); b.setBackground(bg); b.setForeground(Color.WHITE);
    b.setOpaque(true); b.setBorderPainted(false);
    b.setFocusPainted(false); b.setFont(new Font("Segoe UI",Font.BOLD,12)); return
b;
}

static class UserItem    { final int id; final String l; UserItem(int i,String
l){id=i;this.l=l;} public String toString(){return l;} }
static class VehicleItem { final int id; final String l; VehicleItem(int i,String
l){id=i;this.l=l;} public String toString(){return l;} }
static class ZoneItem    { final int id; final String l; ZoneItem(int i,String
l){id=i;this.l=l;} public String toString(){return l;} }
static class SlotItem    { final int id; final String l; SlotItem(int i,String
l){id=i;this.l=l;} public String toString(){return l;} }
}

```

EntryExitPanel.java

```

package parking.ui;

import parking.dao.*;
import parking.model.*;

import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.*;
import java.awt.*;
import java.math.BigDecimal;

```



```

import java.sql.SQLException;
import java.util.List;

/**
 * EntryExitPanel - record vehicle entry (via stored procedure CallableStatement)
 * and exit (calculates fee). Also shows active sessions and full history.
 */
public class EntryExitPanel extends JPanel {

    private final MainFrame parent;
    private final SessionDAO sessionDAO = new SessionDAO();
    private final VehicleDAO vehicleDAO = new VehicleDAO();
    private final ParkingZoneDAO zoneDAO = new ParkingZoneDAO();
    private final ParkingSlotDAO slotDAO = new ParkingSlotDAO();
    private final PaymentDAO paymentDAO = new PaymentDAO();

    // Entry widgets
    private JTextField tfEntryPlate;
    private JComboBox<ZoneItem> cbEntryZone;
    private JComboBox<SlotItem> cbEntrySlot;
    private JButton btnEntry, btnReloadSlots;
    private JLabel lblVehicleInfo;

    // Exit widgets
    private JTextField tfExitPlate;
    private JLabel lblSessionInfo, lblFee;
    private JComboBox<String> cbPayMode;
    private JButton btnLookup, btnExit;
    private ParkingSession currentSession = null;

    // Tables
    private JTable activeTable, historyTable;
    private DefaultTableModel activeModel, historyModel;

    public EntryExitPanel(MainFrame parent) {
        this.parent = parent;
        setLayout(new BorderLayout(6, 6));
        setBorder(new EmptyBorder(8, 8, 8, 8));

        JSplitPane topSplit = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT,
        buildEntryPanel(), buildExitPanel());
        topSplit.setResizeWeight(0.5);

        JSplitPane mainSplit = new JSplitPane(JSplitPane.VERTICAL_SPLIT, topSplit,
        buildSessionTables());
        mainSplit.setResizeWeight(0.4);

        add(mainSplit, BorderLayout.CENTER);
        refresh();
    }

    // — Entry Panel —————
    private JPanel buildEntryPanel() {
        JPanel p = new JPanel(new GridBagLayout());
        p.setBorder(new TitledBorder("Vehicle Entry"));
        GridBagConstraints gc = gbc();

        tfEntryPlate = new JTextField(14);
        cbEntryZone = new JComboBox<>();
        cbEntrySlot = new JComboBox<>();
        lblVehicleInfo = new JLabel(" ");
        lblVehicleInfo.setForeground(new Color(0,100,0));
        btnReloadSlots = btn("Find Vehicle / Reload Slots", new Color(0,102,204));
    }

```

```

        btnEntry          = btn("Record Entry", new Color(34,139,34));

        row(p,gc,0,"License Plate*:", tfEntryPlate);
        row(p,gc,1,"Zone*:",          cbEntryZone);
        row(p,gc,2,"Available Slot*:",cbEntrySlot);
        gc.gridx=0;gc.gridy=3;gc.gridwidth=4; p.add(lblVehicleInfo,gc);
        gc.gridy=4;
        JPanel btns=new JPanel(new FlowLayout(FlowLayout.LEFT,6,0));
        btns.add(btnReloadSlots); btns.add(btnEntry); p.add(btns,gc);

        cbEntryZone.addActionListener(e -> loadAvailableSlots());
        btnReloadSlots.addActionListener(e -> lookupVehicleForEntry());
        btnEntry.addActionListener(e -> recordEntry());

        return p;
    }

    private void lookupVehicleForEntry() {
        String plate = tfEntryPlate.getText().trim().toUpperCase();
        if (plate.isEmpty()) { parent.showError("Enter a license plate."); return; }
        try {
            Vehicle v = vehicleDAO.getVehicleByPlate(plate);
            if (v == null) { parent.showError("No vehicle found with plate: "+plate);
                lblVehicleInfo.setText(" "); return; }
            lblVehicleInfo.setText("Owner: "+v.getOwnerName()+" | Type:
"+v.getVehicleType());
            loadAvailableSlots();
        } catch (SQLException ex) { parent.showError(ex.getMessage()); }
    }

    private void loadAvailableSlots() {
        cbEntrySlot.removeAllItems();
        ZoneItem zi = (ZoneItem) cbEntryZone.getSelectedItem();
        if (zi == null) return;
        try {
            for (ParkingSlot s : slotDAO.getAvailableSlots(zi.id))
                cbEntrySlot.addItem(new SlotItem(s.getSlotId(), s.getSlotNumber()+"
["+s.getSlotType()+"]"));
        } catch (SQLException ex) { parent.showError(ex.getMessage()); }
    }

    private void recordEntry() {
        String plate = tfEntryPlate.getText().trim().toUpperCase();
        if (plate.isEmpty()) { parent.showError("Enter a license plate.");
            return; }
        if (cbEntrySlot.getItemCount()==0){ parent.showError("No available slot
selected."); return; }
        try {
            Vehicle v = vehicleDAO.getVehicleByPlate(plate);
            if (v == null) { parent.showError("Vehicle not registered. Register it
first."); return; }
            // Check if already parked
            ParkingSession active = sessionDAO.getActiveSessionByPlate(plate);
            if (active != null) { parent.showError("Vehicle is already parked (Session
"+active.getSessionId()+")."); return; }

            SlotItem si = (SlotItem) cbEntrySlot.getSelectedItem();
            int sessionId = sessionDAO.recordEntry(v.getVehicleId(), si.id,
v.getUserId());
            parent.setStatus("Entry recorded. Session ID = " + sessionId);
            JOptionPane.showMessageDialog(parent,
                "Entry recorded successfully!\nSession ID: "+sessionId+"\nVehicle:
"+plate+"\nSlot: "+si.label,
                "Entry Recorded", JOptionPane.INFORMATION_MESSAGE);

```

```

        tfEntryPlate.setText(""); lblVehicleInfo.setText(" ");
        refresh();
    } catch (SQLException ex) { parent.showError("Entry
failed:\n"+ex.getMessage()); }
    }

// --- Exit Panel -----
private JPanel buildExitPanel() {
    JPanel p = new JPanel(new GridBagLayout());
    p.setBorder(new TitledBorder("Vehicle Exit & Fee Collection"));
    GridBagConstraints gc = gbc();

    tfExitPlate = new JTextField(14);
    lblSessionInfo = new JLabel("Session: -");
    lblFee = new JLabel("Fee: -");
    lblFee.setFont(new Font("Segoe UI",Font.BOLD,14));
    lblFee.setForeground(new Color(180,0,0));
    cbPayMode = new JComboBox<>(new String[]{"Cash","Card","UPI","Online"});
    btnLookup = btn("Lookup Active Session", new Color(0,102,204));
    btnExit = btn("Record Exit & Pay", new Color(34,139,34));
    btnExit.setEnabled(false);

    row(p,gc,0,"License Plate:", tfExitPlate);
    gc.gridx=0;gc.gridy=1;gc.gridwidth=4; p.add(lblSessionInfo,gc);
    gc.gridy=2; p.add(lblFee,gc);
    gc.gridy=3;gc.gridwidth=2; p.add(new JLabel("Payment Mode:"),gc);
    gc.gridx=2; p.add(cbPayMode,gc);
    gc.gridx=0;gc.gridy=4;gc.gridwidth=4;
    JPanel btns=new JPanel(new FlowLayout(FlowLayout.LEFT,6,0));
    btns.add(btnLookup); btns.add(btnExit); p.add(btns,gc);

    btnLookup.addActionListener(e -> lookupSession());
    btnExit.addActionListener(e -> recordExit());

    return p;
}

private void lookupSession() {
    String plate = tfExitPlate.getText().trim().toUpperCase();
    if (plate.isEmpty()) { parent.showError("Enter a license plate."); return; }
    try {
        currentSession = sessionDAO.getActiveSessionByPlate(plate);
        if (currentSession == null) {
            lblSessionInfo.setText("No active session for: " + plate);
            lblFee.setText("Fee: -"); btnExit.setEnabled(false); return;
        }
        lblSessionInfo.setText("Session ID: "+currentSession.getSessionId()
            +" | Zone: "+currentSession.getZoneName()
            +" | Slot: "+currentSession.getSlotNumber()
            +" | Entry: "+currentSession.getEntryTime());
        lblFee.setText("Calculating at exit...");
        btnExit.setEnabled(true);
    } catch (SQLException ex) { parent.showError(ex.getMessage()); }
}

private void recordExit() {
    if (currentSession == null) { parent.showError("Lookup a session first.");
return; }
    try {
        BigDecimal fee = sessionDAO.recordExit(currentSession.getSessionId());
        // Record payment
        Payment pay = new Payment();
        pay.setUserId(currentSession.getUserId());
        pay.setSessionId(currentSession.getSessionId());
    }
}

```

```

        pay.setAmount(fee);
        pay.setPaymentMode((String)cbPayMode.getSelectedItem());
        pay.setStatus("Paid");
        paymentDAO.addPayment(pay);

        lblFee.setText("Fee Charged: ₹" + fee);
        JOptionPane.showMessageDialog(parent,
            "Exit recorded!\nVehicle: "+currentSession.getLicensePlate()
            +"\nDuration: calculated\nFee: ₹"+fee
            +"\nPayment Mode: "+cbPayMode.getSelectedItem(),
            "Exit & Payment", JOptionPane.INFORMATION_MESSAGE);
        parent.setStatus("Exit recorded. Fee ₹"+fee+" collected.");
        tfExitPlate.setText(""); lblSessionInfo.setText("Session: -");
        btnExit.setEnabled(false); currentSession = null;
        refresh();
    } catch (SQLException ex) { parent.showError("Exit
failed:\n"+ex.getMessage()); }
}

// — Session Tables —————
private JPanel buildSessionTables() {
    JPanel p = new JPanel(new GridLayout(1,2,6,0));

    // Active sessions
    String[] aCols = {"Session ID","Vehicle","User","Zone","Slot","Entry
Time","Status"};
    activeModel = new DefaultTableModel(aCols,0){public boolean isCellEditable(int
r,int c){return false;}};
    activeTable = new JTable(activeModel);
    styleTable(activeTable);
    JPanel ap = new JPanel(new BorderLayout());
    ap.setBorder(new TitledBorder("Active Sessions"));
    ap.add(new JScrollPane(activeTable));
    p.add(ap);

    // History
    String[] hCols = {"Session
ID","Vehicle","Zone","Slot","Entry","Exit","Hours","Fee (₹)","Status"};
    historyModel = new DefaultTableModel(hCols,0){public boolean
isCellEditable(int r,int c){return false;}};
    historyTable = new JTable(historyModel);
    styleTable(historyTable);
    JPanel hp = new JPanel(new BorderLayout());
    hp.setBorder(new TitledBorder("Parking History"));
    hp.add(new JScrollPane(historyTable));
    p.add(hp);

    return p;
}

private void loadActiveSessions() {
    try {
        activeModel.setRowCount(0);
        for (ParkingSession s : sessionDAO.getActiveSessions())
            activeModel.addRow(new
Object[]{s.getSessionId(),s.getLicensePlate(),s.getUserName(),
s.getZoneName(),s.getSlotNumber(),s.getEntryTime(),s.getStatus()});
    } catch (SQLException ex) { parent.showError(ex.getMessage()); }
}

private void loadHistory() {
    try {
        historyModel.setRowCount(0);

```

```

        for (ParkingSession s : sessionDAO.getAllSessions())
            historyModel.addRow(new
Object[] {s.getSessionId(), s.getLicensePlate(), s.getZoneName(),
s.getSlotNumber(), s.getEntryTime(), s.getExitTime(), s.getDurationHrs(), s.getFeeAmount(),
s.getStatus()});
    } catch (SQLException ex) { parent.showError(ex.getMessage()); }
    }

    private void loadZoneCombo() {
        cbEntryZone.removeAllItems();
        try { for(ParkingZone z : zoneDAO.getAllZones()) cbEntryZone.addItem(new
ZoneItem(z.getZoneId(), z.getZoneName()+" (" +z.getZoneType()+")")); }
        catch (SQLException ex){ parent.showError(ex.getMessage()); }
    }

    public void refresh() { loadZoneCombo(); loadAvailableSlots();
loadActiveSessions(); loadHistory(); }

    // — Shared —————
    private GridBagConstraints gbc() {GridBagConstraints gc=new
GridBagConstraints();gc.insets=new
Insets(5,8,5,8);gc.fill=GridBagConstraints.HORIZONTAL;return gc;}
    private void row(JPanel p,GridBagConstraints gc,int r,String l,JComponent
f){gc.gridx=0;gc.gridy=r;gc.gridwidth=1;gc.weightx=0;p.add(new
JLabel(l,gc);gc.gridx=1;gc.weightx=1;gc.gridwidth=3;p.add(f,gc);}
    private void styleTable(JTable t){t.setRowHeight(22);t.setFont(new Font("Segoe
UI",Font.PLAIN,12));t.getTableHeader().setFont(new Font("Segoe
UI",Font.BOLD,12));t.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);t.setAutoCr
eateRowSorter(true);}
    private JButton btn(String t,Color bg){JButton b=new
JButton(t);b.setBackground(bg);b.setForeground(Color.WHITE);b.setOpaque(true);b.setBor
derPainted(false);b.setFocusPainted(false);b.setFont(new Font("Segoe
UI",Font.BOLD,12));return b;}

    static class ZoneItem{final int id;final String l;ZoneItem(int i,String
l){id=i;this.l=l;}public String toString(){return l;}}
    static class SlotItem{final int id;final String label;SlotItem(int i,String
l){id=i;this.label=l;}public String toString(){return label;}}
}

```

PassPaymentPanel.java

```

package parking.ui;

import parking.dao.*;
import parking.model.*;

import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.*;
import java.awt.*;
import java.math.BigDecimal;
import java.sql.*;
import java.util.List;

/**
 * PassPaymentPanel - issue/cancel parking passes and view all payments.
 * Uses two sub-tabs: Parking Passes and Payment History.
 */
public class PassPaymentPanel extends JPanel {

```

```

private final MainFrame parent;
private final PassDAO passDAO = new PassDAO();
private final PaymentDAO paymentDAO = new PaymentDAO();
private final UserDAO userDAO = new UserDAO();
private final VehicleDAO vehicleDAO = new VehicleDAO();
private final ParkingZoneDAO zoneDAO = new ParkingZoneDAO();

// Pass widgets
private JComboBox<UserItem> cbUser;
private JComboBox<VehicleItem> cbVehicle;
private JComboBox<ZoneItem> cbZone;
private JComboBox<String> cbPassType, cbPayMode;
private JTextField tfStartDate, tfEndDate, tfPassFee;
private JButton btnIssuePass, btnCancelPass, btnExpire;
private JTable passTable;
private DefaultTableModel passModel;
private int selectedPassId = -1;

// Payment widgets
private JTable payTable;
private DefaultTableModel payModel;

public PassPaymentPanel(MainFrame parent) {
    this.parent = parent;
    setLayout(new BorderLayout());
    JTabbedPane tabs = new JTabbedPane();
    tabs.addTab("Parking Passes", buildPassTab());
    tabs.addTab("Payment History", buildPaymentTab());
    add(tabs, BorderLayout.CENTER);
    refresh();
}

// =====
// PARKING PASS TAB
// =====
private JPanel buildPassTab() {
    JPanel panel = new JPanel(new BorderLayout(6,6));
    panel.setBorder(new EmptyBorder(8,8,8,8));

    JPanel form = new JPanel(new GridBagLayout());
    form.setBorder(new TitledBorder("Issue Parking Pass"));
    GridBagConstraints gc = gbc();

    cbUser      = new JComboBox<>();
    cbVehicle   = new JComboBox<>();
    cbZone      = new JComboBox<>();
    cbPassType  = new JComboBox<>(new String[]{"Monthly","Semester","Annual"});
    cbPayMode   = new JComboBox<>(new String[]{"Cash","Card","UPI","Online"});

    java.time.LocalDate today = java.time.LocalDate.now();
    tfStartDate = new JTextField(today.toString(), 12);
    tfEndDate   = new JTextField(today.plusMonths(1).toString(), 12);
    tfPassFee   = new JTextField("500.00", 10);

    row(form,gc,0,"User*:",      cbUser);
    row(form,gc,1,"Vehicle*:",   cbVehicle);
    row(form,gc,2,"Zone*:",      cbZone);
    row(form,gc,3,"Pass Type:",  cbPassType);
    row(form,gc,4,"Start Date (yyyy-MM-dd)*:", tfStartDate);
    row(form,gc,5,"End Date (yyyy-MM-dd)*:",   tfEndDate);
    row(form,gc,6,"Pass Fee (₹)*:", tfPassFee);
    row(form,gc,7,"Payment Mode:", cbPayMode);

    btnIssuePass = btn("Issue Pass",      new Color(34,139,34));

```

```

        btnCancelPass = btn("Cancel Selected", new Color(200,40,40));
        btnExpire      = btn("Auto-Expire Passes",new Color(120,60,180));

        JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT,6,4));
        btns.add(btnIssuePass); btns.add(btnCancelPass); btns.add(btnExpire);
        gc.gridx=0;gc.gridy=8;gc.gridwidth=4; form.add(btns,gc);
        panel.add(form, BorderLayout.NORTH);

        // Table
        String[] cols = {"Pass ID","User","Vehicle","Zone","Type","Start","End","Fee
(₹)","Status","Issued"};
        passModel = new DefaultTableModel(cols,0){public boolean isCellEditable(int
r,int c){return false;}};
        passTable = new JTable(passModel);
        styleTable(passTable);
        passTable.getSelectionModel().addListSelectionListener(e->{
            if(!e.getValueIsAdjusting()){int
row=passTable.getSelectedRow();selectedPassId=row>=0?(int)passModel.getValueAt(row,0):
-1;}
        });
        panel.add(new JScrollPane(passTable), BorderLayout.CENTER);

        cbUser.addActionListener(e -> reloadVehicles());
        cbPassType.addActionListener(e -> autoSetEndDate());
        btnIssuePass.addActionListener(e -> issuePass());
        btnCancelPass.addActionListener(e -> cancelPass());
        btnExpire.addActionListener(e -> autoExpire());

        return panel;
    }

    private void issuePass() {
        if (cbUser.getSelectedIndex()<0) {parent.showError("Select a
user.");return;}
        if (cbVehicle.getSelectedIndex()<0) {parent.showError("Select a
vehicle.");return;}
        if (cbZone.getSelectedIndex()<0) {parent.showError("Select a
zone.");return;}
        Date startDate, endDate; BigDecimal fee;
        try {
            startDate = Date.valueOf(tfStartDate.getText().trim());
            endDate   = Date.valueOf(tfEndDate.getText().trim());
            fee       = new BigDecimal(tfPassFee.getText().trim());
        } catch (Exception ex) {
            parent.showError("Invalid date or fee. Use yyyy-MM-dd and numeric fee.");
        }
        return;
        if (!endDate.after(startDate)){parent.showError("End date must be after start
date.");return;}

        ParkingPass p = new ParkingPass();
        p.setUserId(((UserItem)cbUser.getSelectedItem()).id);
        p.setVehicleId(((VehicleItem)cbVehicle.getSelectedItem()).id);
        p.setZoneId(((ZoneItem)cbZone.getSelectedItem()).id);
        p.setPassType((String)cbPassType.getSelectedItem());
        p.setStartDate(startDate);
        p.setEndDate(endDate);
        p.setPassFee(fee);

        try {
            int passId = passDAO.addPass(p);
            // Record payment
            Payment pay = new Payment();
            pay.setUserId(p.getUserId());

```

```

        pay.setPassId(passId);
        pay.setAmount(fee);
        pay.setPaymentMode((String)cbPayMode.getSelectedItem());
        pay.setStatus("Paid");
        paymentDAO.addPayment(pay);

        parent.setStatus("Pass issued. ID = "+passId);
        JOptionPane.showMessageDialog(parent,"Pass issued successfully!\nPass ID:
"+passId+"\nFee paid: ₹"+fee,"Pass Issued",JOptionPane.INFORMATION_MESSAGE);
        refresh();
    } catch (SQLException ex){ parent.showError("Issue pass
failed:\n"+ex.getMessage()); }
    }

    private void cancelPass() {
        if(selectedPassId<0){parent.showError("Select a pass.");return;}
        int c=JOptionPane.showConfirmDialog(parent,"Cancel this
pass?", "Confirm",JOptionPane.YES_NO_OPTION);
        if(c!=JOptionPane.YES_OPTION) return;
        try{ passDAO.updatePassStatus(selectedPassId,"Cancelled");
parent.setStatus("Pass cancelled."); refresh(); }
        catch(SQLException ex){parent.showError(ex.getMessage());}
    }

    private void autoExpire() {
        try{ passDAO.expireOldPasses(); parent.setStatus("Expired passes updated.");
refresh(); }
        catch(SQLException ex){parent.showError(ex.getMessage());}
    }

    private void autoSetEndDate() {
        try {
            java.time.LocalDate start =
java.time.LocalDate.parse(tfStartDate.getText().trim());
            String type = (String)cbPassType.getSelectedItem();
            java.time.LocalDate end = switch(type) {
                case "Monthly" -> start.plusMonths(1);
                case "Semester" -> start.plusMonths(6);
                case "Annual" -> start.plusYears(1);
                default -> start.plusMonths(1);
            };
            tfEndDate.setText(end.toString());
        } catch (Exception ignored){}
    }

    private void loadPasses() {
        try {
            passModel.setRowCount(0);
            for (ParkingPass p : passDAO.getAllPasses())
                passModel.addRow(new
Object[]{p.getPassId(),p.getUserName(),p.getLicensePlate(),p.getZoneName(),
p.getPassType(),p.getStartDate(),p.getEndDate(),p.getPassFee(),p.getStatus(),p.getIssu
edAt()});
        } catch (SQLException ex){parent.showError(ex.getMessage());}
    }

    // =====
    // PAYMENT HISTORY TAB
    // =====
    private JPanel buildPaymentTab() {
        JPanel panel = new JPanel(new BorderLayout(6,6));
        panel.setBorder(new EmptyBorder(8,8,8,8));
    }

```



```

        JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT,10,4));
        top.add(new JLabel("Payment History"));
        JButton btnRefresh = btn("Refresh", Color.DARK_GRAY);
        btnRefresh.addActionListener(e -> loadPayments());
        top.add(btnRefresh);
        panel.add(top, BorderLayout.NORTH);

        String[] cols = {"Pay ID","User","Session ID","Pass ID","Amount
(₹)","Mode","Date","Status"};
        payModel = new DefaultTableModel(cols,0){public boolean isCellEditable(int
r,int c){return false;}};
        payTable = new JTable(payModel);
        styleTable(payTable);
        panel.add(new JScrollPane(payTable), BorderLayout.CENTER);

        return panel;
    }

    private void loadPayments() {
        try {
            payModel.setRowCount(0);
            for (Payment p : paymentDAO.getAllPayments())
                payModel.addRow(new
Object[]{p.getPaymentId(),p.getUserName(),p.getSessionId(),p.getPassId(),
p.getAmount(),p.getPaymentMode(),p.getPaymentDate(),p.getStatus()});
        } catch (SQLException ex){parent.showError(ex.getMessage());}
    }

    // — Helpers —
    private void reloadVehicles() {
        cbVehicle.removeAllItems();
        UserItem ui=(UserItem)cbUser.getSelectedItem(); if(ui==null) return;
        try { for(Vehicle v:vehicleDAO.getVehiclesByUser(ui.id)) cbVehicle.addItem(new
VehicleItem(v.getVehicleId(),v.getLicensePlate()+" ("+v.getVehicleType()+")"); }
        catch(SQLException ex){parent.showError(ex.getMessage());}
    }

    private void loadCombos() {
        cbUser.removeAllItems(); cbZone.removeAllItems();
        try {
            for(User u:userDAO.getAllUsers()) cbUser.addItem(new
UserItem(u.getUserId(),u.getFullName()+" ("+u.getUserType()+")");
            for(ParkingZone z:zoneDAO.getAllZones()) cbZone.addItem(new
ZoneItem(z.getZoneId(),z.getZoneName()));
        } catch(SQLException ex){parent.showError(ex.getMessage());}
        reloadVehicles();
    }

    public void refresh() { loadCombos(); loadPasses(); loadPayments(); }

    private GridBagConstraints gbc(){GridBagConstraints gc=new
GridBagConstraints();gc.insets=new
Insets(4,8,4,8);gc.fill=GridBagConstraints.HORIZONTAL;return gc;}
    private void row(JPanel p,GridBagConstraints gc,int r,String l,JComponent
f){gc.gridx=0;gc.gridy=r;gc.gridwidth=1;gc.weightx=0;p.add(new
JLabel(l,gc);gc.gridx=1;gc.weightx=1;gc.gridwidth=3;p.add(f,gc);}
    private void styleTable(JTable t){t.setRowHeight(22);t.setFont(new Font("Segoe
UI",Font.PLAIN,12));t.getTableHeader().setFont(new Font("Segoe
UI",Font.BOLD,12));t.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);t.setAutoCr
eateRowSorter(true);}
    private JButton btn(String t,Color bg){JButton b=new
JButton(t);b.setBackground(bg);b.setForeground(Color.WHITE);b.setOpaque(true);b.setBor

```

```

derPainted(false);b.setFocusPainted(false);b.setFont(new Font("Segoe
UI",Font.BOLD,12));return b;}

```

```

    static class UserItem    {final int id;final String l;UserItem(int i,String
l){id=i;this.l=l;}public String toString(){return l;}}
    static class VehicleItem {final int id;final String l;VehicleItem(int i,String
l){id=i;this.l=l;}public String toString(){return l;}}
    static class ZoneItem    {final int id;final String l;ZoneItem(int i,String
l){id=i;this.l=l;}public String toString(){return l;}}
}

```

ViolationPanel.java

```

package parking.ui;

import parking.dao.VehicleDAO;
import parking.dao.ViolationDAO;
import parking.model.Vehicle;
import parking.model.Violation;

import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.*;
import java.awt.*;
import java.math.BigDecimal;
import java.sql.SQLException;
import java.util.List;

/**
 * ViolationPanel - record, search, update status and delete parking violations.
 */
public class ViolationPanel extends JPanel {

    private final MainFrame parent;
    private final ViolationDAO violationDAO = new ViolationDAO();
    private final VehicleDAO vehicleDAO = new VehicleDAO();

    private JTextField tfPlate, tfType, tfDescription, tfFine, tfSearch;
    private JComboBox<String> cbStatus;
    private JLabel lblVehicleInfo;
    private JButton btnAdd, btnMarkPaid, btnMarkWaived, btnDelete, btnSearch,
btnShowAll;

    private JTable table;
    private DefaultTableModel tableModel;
    private int selectedViolationId = -1;

    private static final String[] VIOLATION_TYPES = {
        "No Parking Permit", "Wrong Zone", "Expired Pass", "Blocking Pathway",
        "Unauthorized Spot", "Overstay", "Double Parking", "Fire Lane Violation",
        "Other"
    };

    public ViolationPanel(MainFrame parent) {
        this.parent = parent;
        setLayout(new BorderLayout(6, 6));
        setBorder(new EmptyBorder(8, 8, 8, 8));
        add(buildForm(), BorderLayout.NORTH);
        add(buildTable(), BorderLayout.CENTER);
        refresh();
    }

```

```

private JPanel buildForm() {
    JPanel form = new JPanel(new GridBagLayout());
    form.setBorder(new TitledBorder("Record Violation"));
    GridBagConstraints gc = gbc();

    tfPlate      = new JTextField(14);
    lblVehicleInfo = new JLabel(" ");
    lblVehicleInfo.setForeground(new Color(0, 100, 0));

    JComboBox<String> cbType = new JComboBox<>(VIOLATION_TYPES);
    cbType.setEditable(true);
    tfType      = new JTextField(20);
    tfDescription = new JTextField(30);
    tfFine      = new JTextField("500.00", 10);
    cbStatus     = new JComboBox<>(new String[]{"Pending", "Paid", "Waived"});

    // Row 0 - plate + lookup
    gc.gridx=0;gc.gridy=0;gc.weightx=0; form.add(new JLabel("License
Plate*:"),gc);
    gc.gridx=1;gc.weightx=1; form.add(tfPlate,gc);
    JButton btnLookup = makeBtn("Lookup", new Color(0,102,204));
    gc.gridx=2;gc.weightx=0; form.add(btnLookup,gc);
    gc.gridx=3;gc.weightx=1; form.add(lblVehicleInfo,gc);

    // Row 1 - violation type
    gc.gridx=0;gc.gridy=1;gc.weightx=0; form.add(new JLabel("Violation
Type*:"),gc);
    gc.gridx=1;gc.weightx=1;gc.gridwidth=3; form.add(cbType,gc);

    // Row 2 - description
    gc.gridx=0;gc.gridy=2;gc.gridwidth=1;gc.weightx=0; form.add(new
JLabel("Description:"),gc);
    gc.gridx=1;gc.weightx=1;gc.gridwidth=3; form.add(tfDescription,gc);

    // Row 3 - fine + status
    gc.gridx=0;gc.gridy=3;gc.gridwidth=1;gc.weightx=0; form.add(new JLabel("Fine
Amount (₹)*:"),gc);
    gc.gridx=1;gc.weightx=1; form.add(tfFine,gc);
    gc.gridx=2;gc.weightx=0; form.add(new JLabel("Status:"),gc);
    gc.gridx=3;gc.weightx=1; form.add(cbStatus,gc);

    // Row 4 - action buttons
    btnAdd      = makeBtn("Add Violation", new Color(200,40,40));
    btnMarkPaid  = makeBtn("Mark Paid", new Color(34,139,34));
    btnMarkWaived = makeBtn("Mark Waived", new Color(180,130,0));
    btnDelete    = makeBtn("Delete", Color.DARK_GRAY);

    JPanel actionBtns = new JPanel(new FlowLayout(FlowLayout.LEFT,6,0));
    actionBtns.add(btnAdd); actionBtns.add(btnMarkPaid);
    actionBtns.add(btnMarkWaived); actionBtns.add(btnDelete);
    gc.gridx=0;gc.gridy=4;gc.gridwidth=4; form.add(actionBtns,gc);

    // Row 5 - search
    JPanel searchBar = new JPanel(new FlowLayout(FlowLayout.LEFT,6,2));
    searchBar.add(new JLabel("Search (plate / type):"));
    tfSearch = new JTextField(20);
    btnSearch = makeBtn("Go", new Color(0,102,204));
    btnShowAll = makeBtn("Show All", Color.DARK_GRAY);
    searchBar.add(tfSearch); searchBar.add(btnSearch); searchBar.add(btnShowAll);
    gc.gridx=0;gc.gridy=5;gc.gridwidth=4; form.add(searchBar,gc);

    // Events

```

```

        btnLookup.addActionListener(e ->
lookupVehicle(tfPlate.getText().trim().toUpperCase()));
        btnAdd.addActionListener(e -> addViolation(cbType));
        btnMarkPaid.addActionListener(e -> updateStatus("Paid"));
        btnMarkWaived.addActionListener(e -> updateStatus("Waived"));
        btnDelete.addActionListener(e -> deleteViolation());
        btnSearch.addActionListener(e -> searchViolations());
        btnShowAll.addActionListener(e -> { tfSearch.setText(""); loadViolations();
});

        tfSearch.addActionListener(e -> searchViolations());

        return form;
    }

    private JScrollPane buildTable() {
        String[] cols = {"ID", "License Plate", "Slot", "Type", "Description", "Fine
(₹)", "Date", "Status"};
        tableModel = new DefaultTableModel(cols, 0) {public boolean isCellEditable(int
r, int c) {return false;}};
        table = new JTable(tableModel) {
            public Component prepareRenderer(TableCellRenderer r, int row, int col) {
                Component c = super.prepareRenderer(r, row, col);
                if(!isRowSelected(row)) {
                    String s = str(tableModel.getValueAt(row, 7));
                    switch(s) {
break;
                        case "Pending": c.setBackground(new Color(255, 230, 230));
break;
                        case "Paid":      c.setBackground(new Color(230, 255, 230));
break;
                        case "Waived":   c.setBackground(new Color(230, 230, 255));
break;
                        default:         c.setBackground(Color.WHITE);
                    }
                }
                return c;
            }
        };
        table.setRowHeight(22);
        table.setFont(new Font("Segoe UI", Font.PLAIN, 12));
        table.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 12));
        table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        table.setAutoCreateRowSorter(true);
        table.getSelectionModel().addListSelectionListener(e -> {
            if(!e.getValueIsAdjusting()) {int
row=table.getSelectedRow();selectedViolationId=row>=0?(int)tableModel.getValueAt(row, 0
):-1;}
            });
        return new JScrollPane(table);
    }

    private void lookupVehicle(String plate) {
        if(plate.isEmpty()) return;
        try {
            Vehicle v = vehicleDAO.getVehicleByPlate(plate);
            if(v==null){lblVehicleInfo.setText("Vehicle not found.");return;}
            lblVehicleInfo.setText("Owner: "+v.getOwnerName()+" | Type:
"+v.getVehicleType());
        } catch(SQLException ex) {parent.showError(ex.getMessage());}
    }

    private void addViolation(JComboBox<String> cbType) {
        String plate = tfPlate.getText().trim().toUpperCase();
        String type = str(cbType.getSelectedItem()).trim();
        if(plate.isEmpty()){parent.showError("Enter a license plate.");return;}
        if(type.isEmpty()) {parent.showError("Select a violation type.");return;}
    }

```

```

        BigDecimal fine;
        try { fine = new BigDecimal(tfFine.getText().trim()); }
        catch(NumberFormatException e){parent.showError("Invalid fine
amount.");return;}

        try {
            Vehicle v = vehicleDAO.getVehicleByPlate(plate);
            if(v==null){parent.showError("Vehicle not registered.");return;}
            Violation viol = new Violation();
            viol.setVehicleId(v.getVehicleId());
            viol.setViolationType(type);
            viol.setDescription(tfDescription.getText().trim());
            viol.setFineAmount(fine);
            if(violationDAO.addViolation(viol)){
                parent.setStatus("Violation recorded."); clearForm();
loadViolations();
            }
        } catch(SQLException ex){parent.showError("Add violation
failed:\n"+ex.getMessage());}
    }

    private void updateStatus(String status) {
        if(selectedViolationId<0){parent.showError("Select a violation.");return;}

try{violationDAO.updateViolationStatus(selectedViolationId,status);parent.setStatus("V
iolation status: "+status);loadViolations();}
        catch(SQLException ex){parent.showError(ex.getMessage());}
    }

    private void deleteViolation() {
        if(selectedViolationId<0){parent.showError("Select a violation.");return;}
        int c=JOptionPane.showConfirmDialog(parent,"Delete this violation
record?","Confirm",JOptionPane.YES_NO_OPTION);
        if(c!=JOptionPane.YES_OPTION) return;

try{violationDAO.deleteViolation(selectedViolationId);parent.setStatus("Violation
deleted.");loadViolations();}
        catch(SQLException ex){parent.showError(ex.getMessage());}
    }

    private void searchViolations() {
        String kw = tfSearch.getText().trim();
        if(kw.isEmpty()){loadViolations();return;}
        try {
            List<Violation> list = violationDAO.searchViolations(kw);
            populateTable(list);
            parent.setStatus(list.size()+" violation(s) found.");
        } catch(SQLException ex){parent.showError(ex.getMessage());}
    }

    private void loadViolations() {
        try { populateTable(violationDAO.getAllViolations()); }
        catch(SQLException ex){parent.showError(ex.getMessage());}
    }

    private void populateTable(List<Violation> list) {
        tableModel.setRowCount(0);
        for(Violation v:list)
            tableModel.addRow(new
Object[]{v.getViolationId(),v.getLicensePlate(),v.getSlotNumber(),
v.getViolationType(),v.getDescription(),v.getFineAmount(),v.getViolationDate(),v.getSt
atus()});
    }

```

```

private void clearForm() {
    tfPlate.setText(""); tfDescription.setText(""); tfFine.setText("500.00");
    lblVehicleInfo.setText(" "); cbStatus.setSelectedIndex(0);
    table.clearSelection(); selectedViolationId=-1;
}

public void refresh() { loadViolations(); }

private GridBagConstraints gbc(){GridBagConstraints gc=new
GridBagConstraints();gc.insets=new
Insets(4,8,4,8);gc.fill=GridBagConstraints.HORIZONTAL;return gc;}
private JButton makeBtn(String t,Color bg){JButton b=new
JButton(t);b.setBackground(bg);b.setForeground(Color.WHITE);b.setOpaque(true);b.setBorderPainted(false);b.setFocusPainted(false);b.setFont(new Font("Segoe
UI",Font.BOLD,12));return b;}
private String str(Object o){return o!=null?o.toString():"";}
}

```

ReportsPanel.java

```

package parking.ui;

import parking.db.DBConnection;

import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.*;
import java.awt.*;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * ReportsPanel - generates zone-wise, vehicle-wise, occupancy,
 * utilization and revenue reports using plain Statement and PreparedStatement.
 */
public class ReportsPanel extends JPanel {

    private final MainFrame parent;

    private JTable reportTable;
    private DefaultTableModel reportModel;
    private JTextArea summaryArea;
    private JLabel lblReportTitle;

    public ReportsPanel(MainFrame parent) {
        this.parent = parent;
        setLayout(new BorderLayout(6, 6));
        setBorder(new EmptyBorder(8, 8, 8, 8));
        add(buildButtonPanel(), BorderLayout.NORTH);
        add(buildCenter(), BorderLayout.CENTER);
    }

    // — Button panel —————
    private JPanel buildButtonPanel() {
        JPanel p = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 8));
        p.setBorder(new TitledBorder("Generate Reports"));

        String[][] reports = {
            {"Zone-wise Occupancy", "zone_occupancy"},

```

```

        {"Vehicle-wise History", "vehicle_history"},
        {"Revenue Report", "revenue"},
        {"Active Sessions", "active_sessions"},
        {"Utilization %", "utilization"},
        {"Violation Summary", "violations_summary"},
        {"Pass Status Report", "pass_status"},
        {"Daily Revenue", "daily_revenue"}
    };

    for (String[] r : reports) {
        JButton btn = new JButton(r[0]);
        btn.setBackground(new Color(0, 80, 160));
        btn.setForeground(Color.WHITE);
        btn.setOpaque(true);
        btn.setBorderPainted(false);
        btn.setFocusPainted(false);
        btn.setFont(new Font("Segoe UI", Font.BOLD, 12));
        final String key = r[1];
        btn.addActionListener(e -> runReport(key, r[0]));
        p.add(btn);
    }
    return p;
}

private JPanel buildCenter() {
    JPanel p = new JPanel(new BorderLayout(6, 6));

    lblReportTitle = new JLabel("Select a report above to generate.");
    lblReportTitle.setFont(new Font("Segoe UI", Font.BOLD, 14));
    lblReportTitle.setForeground(new Color(0, 60, 120));
    p.add(lblReportTitle, BorderLayout.NORTH);

    reportModel = new DefaultTableModel();
    reportTable = new JTable(reportModel);
    reportTable.setRowHeight(22);
    reportTable.setFont(new Font("Segoe UI", Font.PLAIN, 12));
    reportTable.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 12));
    reportTable.setAutoCreateRowSorter(true);

    summaryArea = new JTextArea(5, 40);
    summaryArea.setEditable(false);
    summaryArea.setFont(new Font("Consolas", Font.PLAIN, 13));
    summaryArea.setBorder(new TitledBorder("Summary"));

    JSplitPane split = new JSplitPane(JSplitPane.VERTICAL_SPLIT,
        new JScrollPane(reportTable), new JScrollPane(summaryArea));
    split.setResizeWeight(0.75);
    p.add(split, BorderLayout.CENTER);

    return p;
}

// — Report dispatcher —————
private void runReport(String key, String title) {
    lblReportTitle.setText("Report: " + title);
    reportModel.setRowCount(0); reportModel.setColumnCount(0);
    summaryArea.setText("");
    try {
        switch (key) {
            case "zone_occupancy" -> zoneOccupancyReport();
            case "vehicle_history" -> vehicleHistoryReport();
            case "revenue" -> revenueReport();
            case "active_sessions" -> activeSessionsReport();
            case "utilization" -> utilizationReport();
        }
    }
}

```

```

        case "violations_summary"-> violationsSummaryReport();
        case "pass_status"      -> passStatusReport();
        case "daily_revenue"    -> dailyRevenueReport();
    }
    parent.setStatus("Report generated: " + title);
} catch (SQLException ex) {
    parent.showError("Report error:\n" + ex.getMessage());
}
}

// — Zone-wise Occupancy —————
private void zoneOccupancyReport() throws SQLException {
    String sql = ""
        SELECT z.zone_name, z.zone_type, z.total_slots,
            COUNT(CASE WHEN s.status='Available' THEN 1 END) AS available,
            COUNT(CASE WHEN s.status='Occupied' THEN 1 END) AS occupied,
            COUNT(CASE WHEN s.status='Reserved' THEN 1 END) AS reserved,
            COUNT(CASE WHEN s.status='Maintenance' THEN 1 END) AS maintenance
        FROM parking_zones z LEFT JOIN parking_slots s ON z.zone_id=s.zone_id
        GROUP BY z.zone_id, z.zone_name, z.zone_type, z.total_slots
        ORDER BY z.zone_name
    "";
    fillTable(sql, new
String[]{"Zone","Type","Total","Available","Occupied","Reserved","Maintenance"});
    buildZoneOccupancySummary();
}

private void buildZoneOccupancySummary() throws SQLException {
    String sql = "SELECT SUM(total_slots) AS t, (SELECT COUNT(*) FROM
parking_slots WHERE status='Available') AS a, (SELECT COUNT(*) FROM parking_slots
WHERE status='Occupied') AS o FROM parking_zones";
    try (Statement st = DBConnection.getConnection().createStatement(); ResultSet
rs = st.executeQuery(sql)) {
        if (rs.next()) {
            int t=rs.getInt("t"), a=rs.getInt("a"), o=rs.getInt("o");
            double pct = t>0 ? (o*100.0/t) : 0;
            summaryArea.setText(String.format(
                "Total Slots: %d\nAvailable: %d\nOccupied: %d\nOccupancy Rate:
%.1f%%", t, a, o, pct));
        }
    }
}

// — Vehicle-wise History —————
private void vehicleHistoryReport() throws SQLException {
    String sql = ""
        SELECT v.license_plate, v.vehicle_type, u.full_name AS owner,
            COUNT(ps.session_id) AS total_sessions,
            COALESCE(SUM(ps.duration_hrs),0) AS total_hours,
            COALESCE(SUM(ps.fee_amount),0) AS total_fee
        FROM vehicles v
        JOIN users u ON v.user_id=u.user_id
        LEFT JOIN parking_sessions ps ON v.vehicle_id=ps.vehicle_id
        GROUP BY v.vehicle_id, v.license_plate, v.vehicle_type, u.full_name
        ORDER BY total_sessions DESC
    "";
    fillTable(sql, new String[]{"License Plate","Type","Owner","Sessions","Total
Hrs","Total Fee (₹)"});
    String sumSql = "SELECT COUNT(*) AS vehicles, SUM(duration_hrs) AS hrs,
SUM(fee_amount) AS fee FROM parking_sessions";
    try (Statement st = DBConnection.getConnection().createStatement(); ResultSet
rs = st.executeQuery(sumSql)) {
        if (rs.next())
            summaryArea.setText(String.format("Total Sessions: -\nTotal Hours:
%.2f\nTotal Revenue: ₹%.2f",
                rs.getDouble("hrs"), rs.getDouble("fee")));
    }
}

```



```

    }
}

// --- Revenue Report -----
private void revenueReport() throws SQLException {
    String sql = ""
        SELECT z.zone_name, z.zone_type,
            COUNT(ps.session_id) AS sessions,
            COALESCE(SUM(ps.fee_amount),0) AS session_revenue,
            (SELECT COALESCE(SUM(pp.pass_fee),0) FROM parking_passes pp WHERE
pp.zone_id=z.zone_id AND pp.status<>'Cancelled') AS pass_revenue
        FROM parking_zones z
        LEFT JOIN parking_slots sl ON z.zone_id=sl.zone_id
        LEFT JOIN parking_sessions ps ON sl.slot_id=ps.slot_id AND
ps.status='Completed'
        GROUP BY z.zone_id, z.zone_name, z.zone_type
        ORDER BY session_revenue DESC
    """;

    fillTable(sql, new String[]{"Zone", "Type", "Sessions", "Session Revenue
(₹)", "Pass Revenue (₹)"});
    String sumSql = "SELECT COALESCE(SUM(amount),0) AS total FROM payments WHERE
status='Paid'";
    try (Statement st = DBConnection.getConnection().createStatement(); ResultSet
rs = st.executeQuery(sumSql)) {
        if (rs.next()) summaryArea.setText("Total Revenue Collected: ₹" +
rs.getDouble("total"));
    }
}

// --- Active Sessions -----
private void activeSessionsReport() throws SQLException {
    String sql = ""
        SELECT ps.session_id, v.license_plate, v.vehicle_type, u.full_name,
            z.zone_name, sl.slot_number, ps.entry_time,
            TIMESTAMPDIFF(MINUTE, ps.entry_time, NOW()) AS minutes_parked
        FROM parking_sessions ps
        JOIN vehicles v ON ps.vehicle_id=v.vehicle_id
        JOIN users u ON ps.user_id=u.user_id
        JOIN parking_slots sl ON ps.slot_id=sl.slot_id
        JOIN parking_zones z ON sl.zone_id=z.zone_id
        WHERE ps.status='Active'
        ORDER BY ps.entry_time
    """;

    fillTable(sql, new String[]{"Session
ID", "Plate", "Type", "Owner", "Zone", "Slot", "Entry Time", "Mins Parked"});
    long cnt = countQuery("SELECT COUNT(*) FROM parking_sessions WHERE
status='Active'");
    summaryArea.setText("Currently Active Sessions: " + cnt);
}

// --- Utilization % -----
private void utilizationReport() throws SQLException {
    String sql = ""
        SELECT z.zone_name, z.total_slots,
            COUNT(ps.session_id) AS total_sessions,
            COALESCE(SUM(ps.duration_hrs),0) AS total_hrs,
            ROUND(COUNT(CASE WHEN sl.status='Occupied' THEN 1 END)*100.0 /
GREATEST(z.total_slots,1), 1) AS occupancy_pct
        FROM parking_zones z
        LEFT JOIN parking_slots sl ON z.zone_id=sl.zone_id
        LEFT JOIN parking_sessions ps ON sl.slot_id=ps.slot_id
        GROUP BY z.zone_id, z.zone_name, z.total_slots
        ORDER BY occupancy_pct DESC
    """;

    fillTable(sql, new String[]{"Zone", "Total Slots", "Total Sessions", "Total
Hrs", "Occupancy %"});
}

```

```

        summaryArea.setText("Utilization report shows current real-time slot occupancy
percentage per zone.");
    }

    // — Violation Summary —————
    private void violationsSummaryReport() throws SQLException {
        String sql = ""
            SELECT violation_type,
                   COUNT(*) AS count,
                   SUM(fine_amount) AS total_fine,
                   COUNT(CASE WHEN status='Pending' THEN 1 END) AS pending,
                   COUNT(CASE WHEN status='Paid' THEN 1 END) AS paid,
                   COUNT(CASE WHEN status='Waived' THEN 1 END) AS waived
            FROM violations
            GROUP BY violation_type
            ORDER BY count DESC
            "";

        fillTable(sql, new String[]{"Violation Type", "Count", "Total Fine
(₹)", "Pending", "Paid", "Waived"});
        String sumSql = "SELECT COUNT(*) AS c, SUM(fine_amount) AS f, SUM(CASE WHEN
status='Pending' THEN fine_amount ELSE 0 END) AS pending_f FROM violations";
        try (Statement st = DBConnection.getConnection().createStatement(); ResultSet
rs = st.executeQuery(sumSql)) {
            if (rs.next())
                summaryArea.setText(String.format(
                    "Total Violations: %d\nTotal Fines: ₹%.2f\nPending Collection:
₹%.2f",
                    rs.getInt("c"), rs.getDouble("f"), rs.getDouble("pending_f")));
        }

        // — Pass Status Report —————
        private void passStatusReport() throws SQLException {
            String sql = ""
                SELECT z.zone_name, pp.pass_type,
                       COUNT(*) AS total,
                       COUNT(CASE WHEN pp.status='Active' THEN 1 END) AS active,
                       COUNT(CASE WHEN pp.status='Expired' THEN 1 END) AS expired,
                       COUNT(CASE WHEN pp.status='Cancelled' THEN 1 END) AS cancelled,
                       COALESCE(SUM(CASE WHEN pp.status='Active' THEN pp.pass_fee ELSE 0
END),0) AS active_revenue
                FROM parking_passes pp
                JOIN parking_zones z ON pp.zone_id=z.zone_id
                GROUP BY z.zone_id, z.zone_name, pp.pass_type
                ORDER BY z.zone_name, pp.pass_type
                "";

            fillTable(sql, new String[]{"Zone", "Pass
Type", "Total", "Active", "Expired", "Cancelled", "Active Revenue (₹)"});
            long active = countQuery("SELECT COUNT(*) FROM parking_passes WHERE
status='Active'");
            summaryArea.setText("Currently Active Passes: " + active);
        }

        // — Daily Revenue —————
        private void dailyRevenueReport() throws SQLException {
            String sql = ""
                SELECT DATE(payment_date) AS date,
                       COUNT(*) AS transactions,
                       SUM(CASE WHEN session_id IS NOT NULL THEN amount ELSE 0 END) AS
session_rev,
                       SUM(CASE WHEN pass_id IS NOT NULL THEN amount ELSE 0 END) AS
pass_rev,
                       SUM(amount) AS total
                FROM payments
                WHERE status='Paid'
                GROUP BY DATE(payment_date)

```

```

        ORDER BY date DESC
        LIMIT 30
        """;
        fillTable(sql, new String[]{"Date","Transactions","Session Rev (₹)","Pass Rev (₹)","Total (₹)"});
        String sumSql = "SELECT SUM(amount) AS t FROM payments WHERE status='Paid'";
        try (Statement st = DBConnection.getConnection().createStatement(); ResultSet
rs = st.executeQuery(sumSql)) {
            if (rs.next()) summaryArea.setText("All-time Total Revenue: ₹" +
rs.getDouble("t"));
        }
    }

    // — Generic table filler —————
    private void fillTable(String sql, String[] columns) throws SQLException {
        reportModel.setColumnCount(0);
        reportModel.setRowCount(0);
        for (String c : columns) reportModel.addColumn(c);

        try (Statement st = DBConnection.getConnection().createStatement();
            ResultSet rs = st.executeQuery(sql)) {
            int cols = rs.getMetaData().getColumnCount();
            while (rs.next()) {
                Object[] row = new Object[cols];
                for (int i = 0; i < cols; i++) row[i] = rs.getObject(i + 1);
                reportModel.addRow(row);
            }
        }
    }

    private long countQuery(String sql) throws SQLException {
        try (Statement st = DBConnection.getConnection().createStatement();
            ResultSet rs = st.executeQuery(sql)) {
            return rs.next() ? rs.getLong(1) : 0;
        }
    }

    public void refresh() { /* reports are on-demand */ }
}

```

PART 5 – OUTPUT / SCREENSHOTS

1. 1. Main Application Window – header, 7 tabs, menu bar, status bar "Connected to campus_parking database"
2. 2. Users & Vehicles Panel – user registration form, search, CRUD table
3. 3. Zones & Slots Panel – colour-coded slot grid (green=Available, red=Occupied, yellow=Reserved)
4. 4. Reservations Panel – booking form with conflict detection
5. 5. Entry / Exit Panel – active sessions table, fee calculation dialog
6. 6. Passes & Payments Panel – pass issuance form and payment history
7. 7. Violations Panel – violation recording form, colour-coded status table
8. 8. Reports Panel – zone-wise occupancy report with summary statistics

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & Vehicles

Zones & Slots

Reservations

Entry / Exit

Passes & Payments

Violations

Reports

Users

Vehicles

User Details:

Full Name*:

Email*:

Phone*:

ID Number*:

User Type:

Student

Address:

Add

Update

Delete

Clear

Search:

Go

Show All

ID	Name	Email	Phone	Type	ID Number	Address	Registered
1	ASDFGH	ASDFGHJ2@	23456789	Student	23	ASXDRCTFVGYBUHJNLK	2026-09-02 02:18:26.0

Connected to campus_parking database.

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & Vehicles

Zones & Slots

Reservations

Entry / Exit

Passes & Payments

Violations

Reports

Parking Zones

Zone Name*:

Zone Type:

Student

Location:

Total Slots:

10

Hourly Rate (₹):

0.00

Description:

Add Zone

Update

Delete

Clear

ID	Zone Name	Type	Location	Total	Available	Rate (₹)	Description
1	Zone-A	Student	Block A - North	50	10	10.00	Student parking near ho...
2	Zone-B	Faculty	Block B - Admin	30	10	0.00	Free faculty parking
3	Zone-C	Visitor	Main Gate	20	10	20.00	Visitor parking
4	Zone-D	Staff	Block D - East	40	10	5.00	Staff parking
5	Zone-E	Service	Service Entrance	10	10	0.00	Service vehicles only
6	Zone-F	General	Central Campus	60	10	15.00	General purpose parking

Parking Slots

Zone*:

Zone-A (Student)

Slot Number*:

Slot Type:

Regular

Status:

Available

Filter Status:

All

Add Slot

Update

Delete

Clear

Filter

ID	Zone	Slot No	Type	Status
6	Zone-A	A01	Handicapped	Available
12	Zone-A	A02	EV	Available
18	Zone-A	A03	Regular	Available
24	Zone-A	A04	Regular	Available
30	Zone-A	A05	Regular	Available
36	Zone-A	A06	Regular	Available
42	Zone-A	A07	Regular	Available
48	Zone-A	A08	Regular	Available
54	Zone-A	A09	Regular	Available
60	Zone-A	A10	Regular	Available

Available

Occupied

Reserved

Maintenance

Connected to campus_parking database.

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & VehiclesZones & SlotsReservationsEntry / ExitPasses & PaymentsViolationsReports

New Reservation

User*:ASDFGH (Student)

Vehicle*:

Zone*:Zone-A - Student

Slot*:A01 [Handicapped]

From (yyyy-MM-dd HH:mm)*:2026-09-02 20:42

Until (yyyy-MM-dd HH:mm)*:2026-09-02 21:42

Available slots: 10

Reserve SlotCancel SelectedRefreshFilter: All

ID	User	License Plate	Zone	Slot	From	Until	Status	Created
----	------	---------------	------	------	------	-------	--------	---------

Connected to campus_parking database.

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & VehiclesZones & SlotsReservationsEntry / ExitPasses & PaymentsViolationsReports

Parking PassesPayment History

Issue Parking Pass

User*:ASDFGH (Student)

Vehicle*:

Zone*:Zone-A

Pass Type:Monthly

Start Date (yyyy-MM-dd)*:2026-09-02

End Date (yyyy-MM-dd)*:2026-10-02

Pass Fee (₹)*:500.00

Payment Mode:Cash

Issue PassCancel SelectedAuto-Expire Passes

Pass ID	User	Vehicle	Zone	Type	Start	End	Fee (₹)	Status	Issued
---------	------	---------	------	------	-------	-----	---------	--------	--------

Connected to campus_parking database.

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & VehiclesZones & SlotsReservationsEntry / ExitPasses & PaymentsViolationsReports

Record Violation

License Plate*:

Violation Type*:

No Parking Permit

Description:

Fine Amount (₹)*:

500.00

 Status:

Pending

Add Violation

Mark Paid

Mark Waived

Delete

Search (plate / type):

Go

Show All

ID	License Plate	Slot	Type	Description	Fine (₹)	Date	Status
----	---------------	------	------	-------------	----------	------	--------

Connected to campus_parking database.

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & VehiclesZones & SlotsReservationsEntry / ExitPasses & PaymentsViolationsReports

Generate Reports

Zone-wise Occupancy

Vehicle-wise History

Revenue Report

Active Sessions

Utilization %

Violation Summary

Pass Status Report

Daily Revenue

Report: Zone-wise Occupancy

Zone	Type	Total	Available	Occupied	Reserved	Maintenance
Zone-A	Student	50	10	0	0	0
Zone-B	Faculty	30	10	0	0	0
Zone-C	Visitor	20	10	0	0	0
Zone-D	Staff	40	10	0	0	0
Zone-E	Service	10	10	0	0	0
Zone-F	General	60	10	0	0	0

Summary

Total Slots: 210

Available: 60

Occupied: 0

Occupancy Rate: 0.0%

Report generated: Zone-wise Occupancy

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & VehiclesZones & SlotsReservationsEntry / ExitPasses & PaymentsViolationsReports

Vehicle Entry

License Plate*:

Zone*:

Zone-A (Student)

Available Slot*:

A01 [Handicapped]

Find Vehicle / Reload Slots

Record Entry

Vehicle Exit & Fee Collection

License Plate*:

Session:--

Fee:--

Payment Mode:

Cash

Lookup Active Session

Record Exit & Pay

Active Sessions

Session ID	Vehicle	User	Zone	Slot	Entry Time	Status
------------	---------	------	------	------	------------	--------

Parking History

Session ID	Vehicle	Zone	Slot	Entry	Exit	Hours	Fee (₹)	Status
------------	---------	------	------	-------	------	-------	---------	--------

Connected to campus_parking database.

30°

Search

ENG IN

92%

08:42 PM02-09-2026

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & VehiclesZones & SlotsReservationsEntry / ExitPasses & PaymentsViolationsReports

UsersVehicles

Vehicle Details

Owner (User)*:

ASDFGH (Student)

License Plate*:

Vehicle Type:

Car

Make:

Model:

Color:

Add

Update

Delete

Clear

Search:

Go

Show All

ID	Owner	License Plate	Type	Make	Model	Color	Registered
----	-------	---------------	------	------	-------	-------	------------

Report generated: Zone-wise Occupancy

30°

Search

ENG IN

90%

08:49 PM02-09-2026

PART 6 – INDIVIDUAL CONTRIBUTION WRITE-UPS

KANIGASRI M (192565010) – Database Design, JDBC & DAO Layer

My contribution to this project focused on the database design, JDBC connectivity, and the complete backend DAO layer. I designed the relational database schema with nine tables — users, vehicles, parking zones, parking slots, reservations, parking sessions, parking passes, payments, and violations — with carefully defined primary key and foreign key relationships to maintain referential integrity across the entire system.

I wrote the two stored procedures: `sp_vehicle_entry`, which atomically records a vehicle session and marks the slot as Occupied, and `sp_vehicle_exit`, which calculates the parking fee based on duration and hourly rate and restores the slot to Available. I implemented the `DBConnection` singleton class managing the JDBC connection lifecycle, and developed six DAO classes — `UserDAO`, `VehicleDAO`, `ParkingZoneDAO`, `ParkingSlotDAO`, `SessionDAO`, and `PaymentDAO` — demonstrating all three required JDBC statement types. Plain Statement was used for simple SELECT all queries, PreparedStatement for all parameterized operations to prevent SQL injection, and CallableStatement in SessionDAO to invoke the stored procedures with IN and OUT parameters.

I tested all SQL queries independently in MySQL Workbench before Java integration, configured the JDBC connection URL parameters, and verified atomic slot status transitions throughout the application. This project gave me deep practical knowledge of relational database normalization, stored procedure design, and secure JDBC programming.

CHAITHIKA L S (192311092) – GUI Design, MainFrame & Entry/Exit Panel

My contribution focused on designing and implementing the graphical user interface, specifically the MainFrame, UserVehiclePanel, and EntryExitPanel. I built the MainFrame as the root application window using BorderLayout, with a dark blue header banner, a JTabbedPane with seven functional tabs, a complete JMenuBar with five menus including keyboard mnemonics and accelerators, and a startup database connectivity test that shows a user-friendly warning dialog when MySQL is unavailable.

I developed the UserVehiclePanel using GridBagLayout with two sub-tabs for user and vehicle management. This panel demonstrates full CRUD operations with input validation — required fields, email format check, license plate uniqueness — search functionality, and table row selection to pre-populate the edit form. I also built the EntryExitPanel, which is operationally the most critical module. The entry side looks up a vehicle by license plate, checks for active sessions, and invokes the stored procedure via CallableStatement. The exit side calculates the fee through the exit stored procedure and records payment automatically.

I also fixed the JComboBox generic type mismatch (UserItem vs String) that caused a compilation error, and applied the `setOpaque(true)` and `setBorderPainted(false)` fix for

buttons to display correctly on Windows. This assignment gave me strong practical experience in event-driven Swing programming, layout management, and end-to-end workflow design.

BABY SHALINI (192521268) – Reservations, Passes, Violations, Reports & Documentation

My role covered the reservation system, parking passes, violations, the reporting module, zone and slot management, and all project documentation. I built the ZoneSlotPanel using a JSplitPane with a custom prepareRenderer that colour-codes slot rows — green for Available, red for Occupied, yellow for Reserved, and grey for Maintenance — giving operators an immediate visual overview. I developed ParkingZoneDAO and ParkingSlotDAO including the getAvailableSlots and getSlotsByStatus methods used across multiple panels.

I built the ReservationPanel with datetime validation, dynamic slot loading, and a conflict detection PreparedStatement overlap query that prevents double-booking. The PassPaymentPanel auto-calculates end dates using Java LocalDate — Monthly adds one month, Semester six months, Annual one year — and records a payment entry for each pass. The ViolationPanel provides record creation, Paid/Waived status updates, deletion, and keyword search with colour-coded rows.

I developed the ReportsPanel with eight report types using complex SQL with GROUP BY, CASE expressions, subqueries, and aggregates. I also handled all documentation — pseudocode for all eleven modules, the README setup guide, and this submission document — and managed the GitHub repository. This project significantly strengthened my skills in analytical SQL, Swing custom rendering, and full-stack Java development.

PART 7 – GITHUB REPOSITORY LINKS

Member	Register No	GitHub Repository Link
KANIGASRI M	192565010	https://github.com/KANIGASRI31/CSA0925-PROGRAMMING-IN-JAVA/tree/main/ASSIGNMENT
CHAITHIKA L S	192311092	https://github.com/chaithika09/Java-Assignment-chaithika
BABY SHALINI	192521268	https://github.com/192521268simats-cell/java-

SDG RELEVANCE

SDG 9 – Industry, Innovation and Infrastructure

The system digitises and automates manual campus parking management, replacing paper-based methods with a structured database-driven Java application, directly contributing to resilient campus infrastructure.

SDG 11 – Sustainable Cities and Communities

Real-time slot availability, reservation management, and utilisation reports reduce vehicle congestion on campus. Drivers no longer circle looking for parking, reducing fuel waste and improving campus environment.

SDG 13 – Climate Action

The system includes dedicated EV (Electric Vehicle) slot types. By tracking and prioritising EV slots, the system encourages electric vehicle adoption on campus, contributing to reduced carbon emissions.